Fact checking con evidenza per frase — implementazione (KnowLine-4B)

2026-11-04

Indice

Fact checking con evidenza per frase — documentazione dell'implementazione **3**
1. Che cosa fa il sistema . . . 3
2. La pipeline . . . 3
3. Notazione . . . 4
4. Stato dell'implementazione (2026-11-04) . . . 5
5. Obiettivi (dal piano) . . . 5

Passo 1 — Il decision model e gli hook per frase **5**
In breve . . . 5
1. Il modello: KnowLine-4B-Gen5.5 su Qwen3.5 . . . 6
2. Caricamento . . . 7
3. Gli hook: riassunto delle frasi . . . 7
4. La riga di attenzione . . . 8
5. `forward_sentences`: che cosa restituisce . . . 9
6. Verifiche (`python -m evidence.model --selftest`) . . . 9
7. Codice . . . 9

Passo 2 — I dati dalle fonti e la divisione in frasi **9**
In breve . . . 9
1. Le fonti . . . 10
2. Il divisore in frasi (`evidence/sentences.py`) . . . 10
3. Il formato di una riga . . . 11

Passo 3 — Dati generati (NON implementato) **11**

Passo 4 — Split per documento e mix di training **12**
In breve . . . 12
1. Assegnazione deterministica per documento . . . 12
2. Le regole per fonte . . . 12
3. I set (prodotti il 2026-11-04) . . . 12
4. Il mix . . . 13
5. Codice . . . 14

Passo 5 — Il prompt di KnowLine e il preambolo **14**
In breve . . . 14
1. Com'è fatto il prompt . . . 15
2. Perché il preambolo . . . 15
3. Come si inserisce senza toccare il codice degli autori . . . 16
4. La scelta del preambolo (`check_preamble.py`) . . . 16
5. Codice . . . 16

Passo 6 — Dalle frasi ai token, e salvataggio delle feature **17**
In breve . . . 17
1. Il problema . . . 17
2. Passo per passo (`render.render`) . . . 17
3. Estrazione (`extract.py`) . . . 18

4. Lettura (`features.FeatureStore`) . . . 19
5. Verifiche . . . 19
6. Codice . . . 19

Passo 7 — Metodi semplici da battere **20**
In breve . . . 20
1. Protocollo . . . 20
2. I metodi . . . 20
3. Prima prova (300 righe di `dev_evid_select`, metà B: 146 righe valutate) . . . 21
4. Codice . . . 21

Passo 8 — L'architettura della testa e la loss **21**
In breve . . . 21
1. Forme dei tensori . . . 23
2. Gli stadi, in formule . . . 23
3. Parametri (KnowLine-4B, $D = 2560$; calcolati da `EvidenceHead`) . . . 24
4. Interruttori e configurazioni . . . 25
5. La loss . . . 25
6. Codice . . . 28

Passo 9 — Training della testa (fase 1a) **28**
In breve . . . 28
1. La configurazione di un esperimento . . . 28
2. Il ciclo (`train`) . . . 29
3. La regola di selezione (`choose_rule`) . . . 29
4. Che cosa scrive una run . . . 29
5. Che cosa guardare . . . 30
6. Controlli prima di ogni nuova configurazione . . . 30
7. Codice . . . 30

Passo 10 — Valutazione e latenza **31**
In breve . . . 31
1. Su quali righe . . . 31
2. Metriche dell'evidenza (`summarize`) . . . 31
3. Metriche del verdetto (`verdict_metrics`) . . . 32
4. Che cosa si salva . . . 32
5. Latenza (`latency.py`) . . . 33
6. Codice . . . 33

Passo 11 — Esperimenti **33**
In breve . . . 33
1. Regole di decisione . . . 33
2. Le configurazioni pronte . . . 34
3. Esperimenti che richiedono altre feature o un altro mix . . . 35
4. Sequenza consigliata . . . 35
5. Che cosa NON si può fare con questa implementazione . . . 35

Passo 12 — Rilascio e server **35**
In breve . . . 35
1. Una richiesta (`Service.answer`) . . . 35
2. La cartella di rilascio (`release.build`) . . . 36
3. Perché il divisore va rilasciato con il modello . . . 36
4. Controlli prima del rilascio . . . 36
5. Codice . . . 37

Comandi in ordine, checklist e problemi frequenti **37**
1. Comandi in ordine . . . 37
2. Checklist . . . 38
3. Se succede... . . . 38

Che cosa non è implementato, e da dove partire **38**
La strada più promettente . . . 39

Riepilogo — tutte le formule in una pagina **39**
1. Dal documento alle feature (un forward) . . . 39
2. La testa (`head.EvidenceHead`) . . . 40
3. Le loss (`losses.py`) . . . 40
4. Training (`train_head.py`) . . . 41
5. Metriche (`metrics.py`, `eval_evidence.py`) . . . 41
6. Dati (`make_splits.py`, `make_mix.py`, `baselines.py`) . . . 41
7. Perché ogni scelta, in una riga . . . 41

Fact checking con evidenza per frase — documentazione dell'implementazione

> Questa cartella documenta il sistema **così come è implementato** in `evidence/` per **KnowLine-4B**. Il piano di partenza, con tutte le varianti (altri modelli, dati generati, fase 2), è in `../../fine_tuning_factcheck_evidence_complete/`. Ogni passo indica i file e le funzioni che lo realizzano. La guida pratica ai comandi è `evidence/code_report.md`.

1. Che cosa fa il sistema

Un **decision model** (KnowLine-4B) riceve un documento e un claim e, con **un solo passaggio sul testo** (un *forward*, senza generare token), dà

$$P(\text{supportato}) = \sigma(\ell_A - \ell_B),$$

dove ℓ_A e ℓ_B sono i log-punteggi delle etichette "A. Yes" e "B. No" all'ultimo token del prompt.

Aggiungiamo una piccola rete, la **testa di evidenza** (3,7 M parametri contro i 4 miliardi del modello). Dagli stati interni calcolati **in quello stesso forward**, assegna a ogni frase k del documento un punteggio s_k e, al documento, una soglia $s_\star$. Le frasi con $s_k > s_\star$ sono l'evidenza:


```
{"p_supported": 0.9987, "supported": true,
 "evidence": [{"sentence": 0, "start": 0,   "end": 77,  "score": 0.83,
               "text": "Saint Paul School of Theology is a seminary located in Kansas City, Missouri."},
              {"sentence": 4, "start": 185, "end": 259, "score": 0.61,
               "text": "Nancy R. Howell is Professor of Theology at Saint Paul School of Theology."}]}
```


(Risposta reale del server, `python -m evidence.serve --demo`, con una testa addestrata su 300 righe. Claim: *"Nancy R. Howell teaches at a seminary in Kansas City."*)

Il caso difficile è l'**evidenza sparsa**: due frasi lontane che dimostrano il claim **solo insieme**. Nell'esempio, la frase 0 non nomina Nancy Howell e la frase 4 non nomina Kansas City.

Tre principi:

1. **Un solo forward.** La testa usa vettori che il modello calcola comunque. Costo misurato: +1,8 ms su circa 340 ms (+0,5%).
2. **Il verdetto non cambia.** Il modello è congelato e si addestra solo la testa.
3. **Le frasi si vedono fra loro.** Nel modello causale la frase 0 non vede la frase 4; nella testa un transformer **non causale** sulle frasi le mette in comunicazione.

2. La pipeline

Figura 1 — I passi e i file che li realizzano. Arancione: richiede il modello sulla GPU. Verde: solo la testa (minuti, anche su CPU).

#	Passo	Documento	Codice
0	Dataset con evidenza annotata	02	`evidence/data/build.py`, `evidence/sentences.py`
1	Split per documento e mix di training	04	`evidence/data/make_splits.py`, `evidence/data/make_mix.py`

#	Passo	Documento	Codice
2	Prompt di KnowLine e posizione delle frasi nei token	05, 06	`evidence/render.py`
3	Forward con gli hook	01	`evidence/model.py`
4	Scelta del preambolo	05	`evidence/check_preamble.py`
5	Estrazione delle feature su disco	06	`evidence/extract.py`, `evidence/features.py`
6	Baseline	07	`evidence/baselines.py`
7	Architettura della testa e loss	08	`evidence/head.py`, `evidence/losses.py`
8	Training	09	`evidence/train_head.py`, `evidence/configs/*.json`
9	Valutazione e latenza	10	`evidence/eval_evidence.py`, `evidence/metrics.py`, `evidence/latency.py`
10	Esperimenti	11	`evidence/configs/`, `evidence/recipes/`
11	Rilascio e server	12	`evidence/release.py`, `evidence/serve.py`
—	Comandi e checklist	13	
—	Cosa non è implementato	14	
—	Tutte le formule in una pagina	15	

Il passo 03 (dati generati) è fuori dal perimetro dell'implementazione.

3. Notazione

Simbolo	Significato	Valore per KnowLine-4B
T	posizione dell'ultimo token del prompt (**posizione di risposta**)	
D	dimensione degli stati nascosti	2.560
$H^{(l)} \in \mathbb{R}^{(T+1)\times D}$	uscita del layer l (un vettore per token)	
$\mathcal{L}$	layer da cui si leggono le frasi	$\{19, 23, 27, 31\}$, $L = \lvert\mathcal{L}\rvert = 4$
$\mathcal{F}$	layer ad attenzione piena	$\{3, 7, 11, \dots, 31\}$ (8 layer)
n	frasi del documento	mediana 46 nel mix, massimo 512
$[a_k, b_k]$	primo e ultimo token della frase k nel prompt	
$n_k = b_k - a_k + 1$	token della frase k	
$h_k^{end}, h_k^{mean} \in \mathbb{R}^{L\times D}$	ultimo token e media dei token della frase k, nei layer di $\mathcal{L}$	
$q \in \mathbb{R}^{L\times D}$	stato della posizione di risposta, nei layer di $\mathcal{L}$	
$att_k \in \mathbb{R}^{128}$	attenzione che la posizione di risposta dà alla frase k (8 layer × 16 teste)	
d	dimensione interna della testa	256
s_k, $s_\star$	punteggio della frase k; soglia del documento (`s0` nel codice)	

Simbolo	Significato	Valore per KnowLine-4B
G	insieme delle frasi gold di una riga (unione degli insiemi alternativi)	
ω	peso delle frasi **non** gold nella loss	0,3–1
δ	spostamento della soglia scelto sul dev	$[-1, 1]$

4. Stato dell'implementazione (2026-11-04)

Parte	Stato
Split e mix	lanciati: `dataset/evidence/splits/`, `dataset/evidence/mixes/ev_head.jsonl` (20.249 righe)
Rendering, hook, forward	verificati sui dati veri (06 §8, 01 §6)
Testa, loss, metriche	test unitari superati (`tests/test_evidence_head.py`)
Estrazione, baseline, training, valutazione, server, latenza	provati end-to-end su 300 righe
Estrazione completa (circa 2,5 h, circa 50 GB), esperimenti	**da lanciare** (13)

I numeri che compaiono in questa documentazione come "prova su 300 righe" dicono solo che la pipeline funziona. I risultati veri richiedono l'estrazione completa.

5. Obiettivi (dal piano)

Obiettivo	Soglia	Come si misura
Verdetto invariato	identico a quello senza testa	`eval_evidence.py`, sezione `verdict`
F1 per frase	almeno 15 punti sopra E-LEX	`baselines.py` contro `eval_evidence.py`
Insieme completo su HoVer	≥ 0,65	`metrics.summarize` → `full_set`, `by_source`
Evidenza lontana	F1 con gold a ≥ 20 frasi di distanza al massimo 5 punti sotto quello delle vicine	`f1_by_sent_distance`
Sufficienza	≥ 85%	`eval_evidence.py --sufficiency`
Latenza	≤ +3% o +2 ms	`latency.py` (misurato: +0,5%)

Passo 1 — Il decision model e gli hook per frase

In breve

Il verdetto e le feature delle frasi escono dallo **stesso forward** di KnowLine-4B. Il codice (`evidence/model.py`) non modifica il modello. Aggiunge due cose:

1. **hook** sui layer 19, 23, 27 e 31: mentre il forward passa, riassumono ogni frase in due vettori (h_k^{end}, h_k^{mean}) e salvano lo stato della posizione di risposta (q);
2. una **funzione di attenzione** registrata in transformers (`sdpa_capture`): fa l'attenzione normale e, negli 8 layer pieni, ricalcola una sola riga di pesi, quella della posizione di risposta verso tutti i token. Da questa riga si ricava quanto il modello "guarda" ogni frase (att_k).

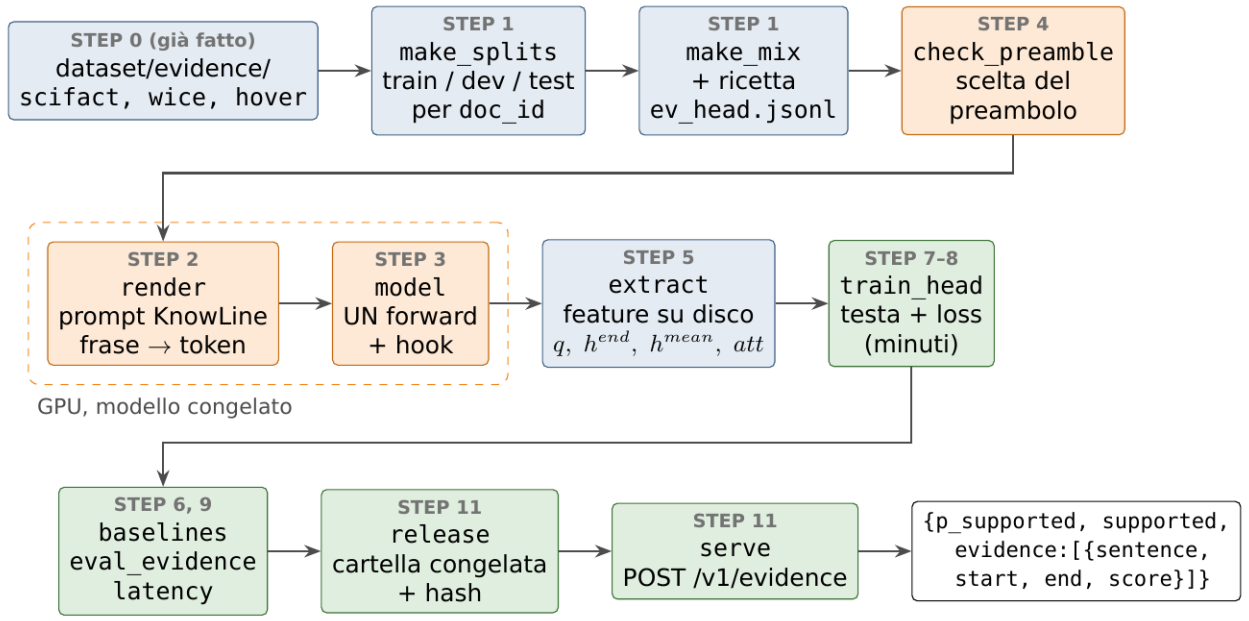


Figura 1: Pipeline

1. Il modello: KnowLine-4B-Gen5.5 su Qwen3.5

KnowLine-4B (`PelaAI/KnowLine-4B-Gen5.5`, revisione `779944d`) è un Qwen3.5-4B fine-tunato dagli autori come decision model. Le impostazioni sono in `evidence/config.py` (`MODEL_REPO`, `MODEL_REVISION`).

Parametro (`config.json` dello snapshot)	Valore
layer	32
dimensione D	2.560
layer ad attenzione piena	3, 7, 11, 15, 19, 23, 27, 31 (uno ogni quattro)
altri 24 layer	Gated DeltaNet (attenzione lineare)
teste di query / di chiavi (layer pieni)	16 / 4, dimensione di testa 256
DeltaNet: teste di valore / di chiave	32 / 16, dimensione 128, convoluzione causale di ampiezza 4
MLP	9.216 (SwiGLU)
vocabolario	248.320

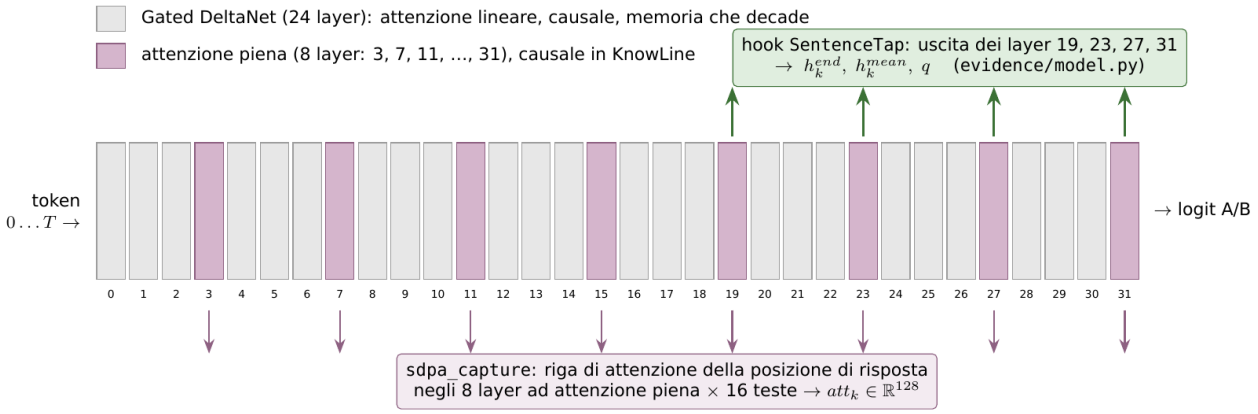


Figura 2: Layer di Qwen3.5

Figura 2 — I 32 layer. In viola i layer ad attenzione piena. Le frecce verdi indicano i layer da cui gli hook leggono le frasi; quelle viola le righe di attenzione catturate.

1.1 Un layer, in formule

Ogni layer aggiorna il **flusso residuo** $H^{(l)} \in \mathbb{R}^{(T+1)\times D}$ (un vettore per token):

$$\tilde{H} = H^{(l-1)} + \text{Mixer}_l\big(\text{RMSNorm}(H^{(l-1)})\big), \qquad H^{(l)} = \tilde{H} + \text{MLP}_l\big(\text{RMSNorm}(\tilde{H})\big).$$

$$S_t = \alpha_t\, S_{t-1}(I - \beta_t\, \mathbf{k}_t\mathbf{k}_t^\top) + \beta_t\, \mathbf{v}_t\mathbf{k}_t^\top, \qquad o_t = S_t\, \mathbf{q}_t,$$

con $\alpha_t = \exp(g_t) \in (0,1)$ (decadimento appreso: la memoria si affievolisce) e $\beta_t = \sigma(\cdot) \in (0,1)$ (forza della scrittura). Anche questo è **causale**, e in più con memoria che decade.

Conseguenze per la testa:

- Quando il modello legge la frase k, non ha ancora visto le frasi successive. È per questo che la testa ha un transformer non causale sulle frasi (08 §3.C).
- Leggere subito **dopo un layer pieno** (19, 23, 27, 31) dà il vettore più aggiornato: ha appena "guardato" esplicitamente tutto il testo precedente, compreso il claim del preambolo (05).

1.2 Il verdetto

All'ultimo token T il modello produce i logit sul vocabolario, $z = W_U\,\mathrm{RMSNorm}(H_T^{(31)})$. Con $\ell = \log\mathrm{softmax}(z)$ e le etichette A ("Yes") e B ("No"):

$$P(\text{supportato}) = \frac{e^{\ell_A}}{e^{\ell_A} + e^{\ell_B}} = \sigma(\ell_A - \ell_B).$$

La temperatura di calibrazione degli autori (1,5) è già inclusa nei pesi. La soglia è 0,5 (`config.VERDICT_THRESHOLD`). Codice: `model.p_supported(answer_logits)`. Gli id delle etichette vengono da `knowline_layout.labels_and_ids`, attraverso `render.frontend()`.

Si calcolano solo i logit dell'ultima posizione (`logits_to_keep=1`), come fa l'adapter del benchmark (`models/knowline-4b/adapter.py`, `hf_score`). I logit di tutte le posizioni occuperebbero circa 0,5 MB per token.

2. Caricamento

`model.load_model(device)` carica il modello come `knowline_server.HF`: classe multimodale di transformers, bf16, `device_map`, con una differenza, `attn_implementation="sdpa_capture"`. Prima di caricarlo, `register_attention()` registra la funzione in `transformers.AttentionInterface`. I pesi sono congelati (`requires_grad_(False)`). `text_layers(model)` trova la `ModuleList` dei 32 layer testuali (il modello ha anche una torre visiva, che si esclude).

Servono `flash-linear-attention` (`fla`) e `causal-conv1d`: transformers 5.17 li usa automaticamente per i layer DeltaNet (`use_kernel_func_from_hub_with_fallback`). Senza, il forward è molto più lento.

3. Gli hook: riassunto delle frasi

Figura 3 — In alto: dall'uscita di un layer si ricavano, per la frase k (token $a_k \dots b_k$), l'ultimo token e la media; q è lo stato della posizione di risposta. In basso: la riga di attenzione del token T, sommata sui token della frase.

Per ogni layer $l \in \mathcal{L} = \{19, 23, 27, 31\}$, un *forward hook* (`SentenceTap._hook`) riceve $H^{(l)}$ e calcola subito, sulla GPU:

$$h^{end}_{k,l} = H^{(l)}_{b_k}, \qquad h^{mean}_{k,l} = \frac{1}{n_k}\sum_{i=a_k}^{b_k} H^{(l)}_i = \frac{C_{b_k+1} - C_{a_k}}{b_k - a_k + 1}, \quad C_t = \sum_{i<t} H^{(l)}_i, \qquad q_l = H^{(l)}_T.$$

- Le somme cumulative C si calcolano una volta sola, in fp32, per tutte le frasi: il costo è $O(TD)$ per layer, indipendente da n.
- Esce un tensore $n \times D$ per layer: i vettori di tutti i token **non vengono mai conservati**.
- **Perché due riassunti.** h^{end} è l'ultimo token: in un modello causale ha letto tutta la frase e ciò che la precede. h^{mean} pesa tutti i token allo stesso modo e non dipende dalla punteggiatura finale. In una frase lunga con il fatto rilevante all'inizio, la media ne conserva di più.

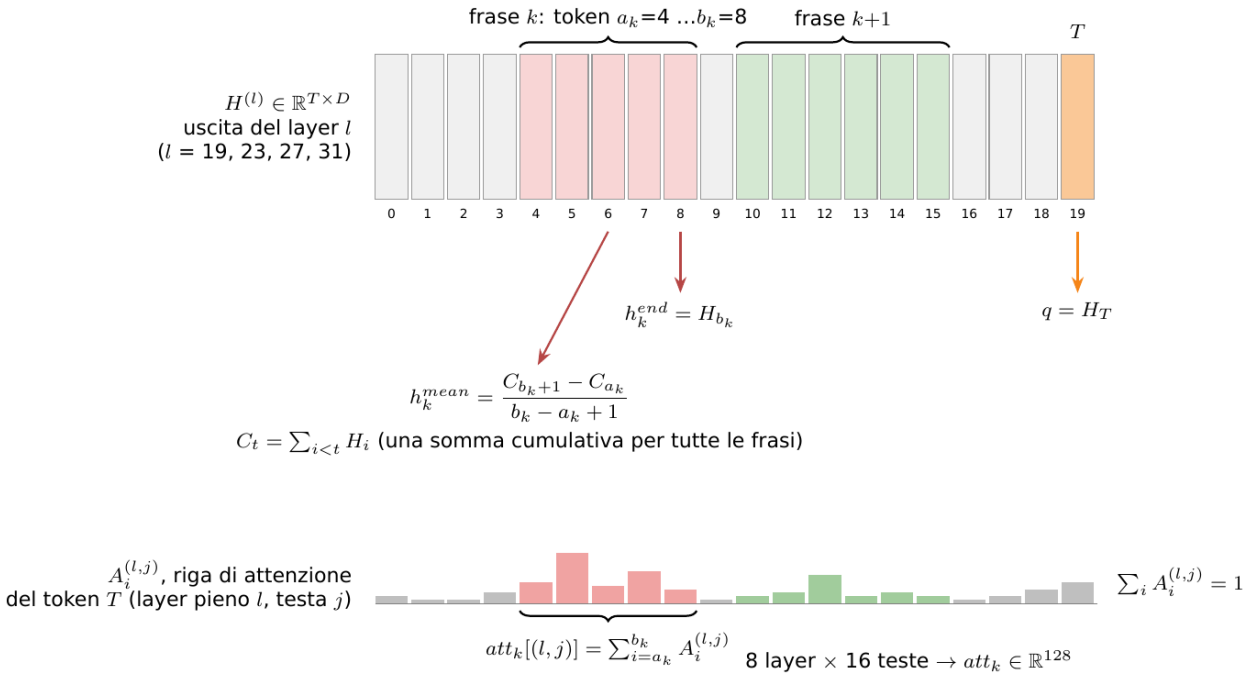


Figura 3: Riassunti delle frasi

- **Perché non solo il layer 31.** L'ultimo layer prepara la predizione del token successivo; nelle posizioni diverse da T può aver già scartato informazione sul contenuto. I layer pieni 19, 23 e 27 la conservano. Quale layer serve lo decide la miscela appresa della testa (08 §3.A).

4. La riga di attenzione

Le implementazioni veloci dell'attenzione (SDPA, flash) non restituiscono i pesi. Chiederli in modalità "eager" costerebbe $16 \times T^2$ numeri per layer: con $T = 16.000$ sono 4 G valori. Ne serve però **una sola riga**: quella della posizione di risposta. `sdpa_capture(module, query, key, value, attention_mask, scaling)`:

1. chiama `sdpa_attention_forward` (l'attenzione normale, che è l'uscita del modello e resta invariata);
2. se il layer è in $\mathcal{F}$, calcola **in fp32**:

$$A_i^{(l,j)} = \text{softmax}_i\left(\mathbf{q}_T^{(l,j)} \cdot \mathbf{k}_i^{(l,\lfloor j/4 \rfloor)} \cdot \tfrac{1}{\sqrt{256}}\right), \qquad i = 0 \dots T,$$

con `key.repeat_interleave(4, dim=1)` per passare dalle 4 teste di chiavi alle 16 di query. Il modello è causale e si lavora con batch 1, quindi l'ultima riga vede tutti i token: non serve maschera. Il codice applica comunque la maschera, se il modello ne passa una; 3. salva $A^{(l,\cdot)} \in \mathbb{R}^{16\times(T+1)}$ in `CAPTURE["rows"][l]`.

Poi `attention_per_sentence(rows, spans)` somma i pesi sui token di ogni frase, sempre con somme cumulative:

$$att_k[(l,j)] = \sum_{i=a_k}^{b_k} A_i^{(l,j)}, \qquad (l,j) \in \mathcal{F} \times \{0,\dots,15\}, \qquad att_k \in \mathbb{R}^{128}.$$

Siccome $\sum_i A_i^{(l,j)} = 1$, ogni componente è la **quota di attenzione** che la testa (l,j) dà alla frase. Le colonne sono ordinate così: layer 3 teste 0–15, layer 7 teste 0–15, …, layer 31.

5. `forward_sentences`: che cosa restituisce

`model.forward_sentences(model, ex, features=True, to_cpu=True)`, dove `ex` è l'uscita di `render.render` (06):

Chiave	Forma	Tipo	Contenuto
`answer_logits`	[2]	fp32	(ℓ_A, ℓ_B)
`q`	[4, D]	fp32	q_l, $l \in \mathcal{L}$
`h_end`	[n, 4, D]	bf16	$h^{end}_{k,l}$
`h_mean`	[n, 4, D]	bf16	$h^{mean}_{k,l}$
`att`	[n, 128]	fp16	att_k
`n_tok`	[n]	int16	n_k

- Con `features=False` si ottiene solo il verdetto: serve per la sufficienza, l'ablazione e la latenza.
- Con `to_cpu=False` i tensori restano sulla GPU: lo usa il server, per evitare la copia.
- Lo stato condiviso fra la funzione di attenzione e `forward_sentences` è il dizionario `CAPTURE`: il modello chiama la funzione di attenzione senza argomenti nostri, quindi non si può passare altro.

6. Verifiche (`python -m evidence.model --selftest`)

Test	Come	Risultato (RTX 5090 Laptop, 10 righe di dev)
Verdetto invariato	P con hook e cattura contro "sdpa" senza hook	$\max\|\Delta P\| = 0$
Riassunti delle frasi	`h_end`/`h_mean` contro `output_hidden_states` (layer 19, 23, 27; nel 31 `hidden_states` ha già la norma finale)	differenza relativa massima 0,0027 (bf16)
Attenzione	att_k contro la modalità "eager" (pesi in bf16) su un prompt di 2.500 token	correlazione 0,9994, differenza massima 0,006

Velocità: un forward su circa 3.100 token richiede circa 0,37 s, e gli hook non aggiungono tempo misurabile. Sono circa 25 TFLOP, quindi è il limite della GPU laptop.

7. Codice

Funzione / classe	File	Ruolo
`sdpa_capture`, `register_attention`, `CAPTURE`	`evidence/model.py`	riga di attenzione della posizione di risposta
`load_model`, `text_layers`	`evidence/model.py`	caricamento come il front end degli autori
`SentenceTap`, `get_tap`	`evidence/model.py`	hook: h^{end}, h^{mean}, q
`attention_per_sentence`	`evidence/model.py`	att_k
`forward_sentences`, `p_supported`	`evidence/model.py`	il forward unico e il verdetto
`selftest`	`evidence/model.py`	le verifiche del §6

Passo 2 — I dati dalle fonti e la divisione in frasi

In breve

Si usano tre dataset pubblici con l'evidenza annotata: **SciFact**, **WiCE** e **HoVer** (il testo delle pagine di HoVer viene da HotpotQA distractor). Ogni fonte viene convertita in uno **stesso**

formato di riga: - il documento è diviso in frasi con il **nostro** divisore; - le frasi di evidenza della fonte sono riportate sulle nostre frasi.

Codice (già presente prima dell'implementazione della testa e invariato): `evidence/data/build.py`, `evidence/data/schema.py`, `evidence/data/sources/` e il divisore `evidence/sentences.py`. Comando:

```
python -m evidence.data.build --source scifact wice hover --hover-db-fallback
```

1. Le fonti

Fonte	Righe	Positivi / negativi	Documento	Frasi per documento (mediana)	Frasi gold per positivo
SciFact	1.258	508 / 750	titolo + abstract biomedico	9	1,8
WiCE	5.294	3.100 / 2.194	pagina web citata da Wikipedia	68	2,6
HoVer	22.063	12.974 / 9.089	circa 10 pagine Wikipedia (evidenza + disturbo)	46	3,0

Etichette per fonte: - **SciFact**: `SUPPORT` → 1; `CONTRADICT` e `NEI` → 0. - **WiCE**: `supported` → 1; `not_supported` → 0; `partially_supported` → 0 senza loss di evidenza (`evidence_term = none`). - **HoVer**: `SUPPORTED` → 1; `NOT_SUPPORTED` → 0.

HoVer. Le pagine mancanti in HotpotQA (claim a 3–4 salti) vengono prese dal database Wikipedia di HoVer, e la frase gold si ritrova con il nostro divisore. Queste righe hanno `meta.gold_alignment = "db_split"` (9.208); nel 2% dei casi la frase gold è sfasata di una. Le altre sono `"exact"` (12.855).

WiCE e LLM-AggreFact. WiCE è anche una fonte del test LLM-AggreFact. `meta.aggrefact_overlap` vale `"dev"` o `"test"` quando la pagina compare lì. Lo split (04) impedisce che queste righe finiscano in training.

2. Il divisore in frasi (`evidence/sentences.py`)

`split(text)` restituisce gli intervalli $[s_k, e_k)$ di caratteri delle frasi. È l'**unica** divisione usata in tutta la pipeline: dati, rendering, training e server. Le regole:

1. **blocchi**: si va a capo dopo una riga vuota, prima di un elemento di elenco e dopo una riga senza punteggiatura finale (un titolo); una riga che finisce con virgola o trattino continua nella successiva;
2. **Punkt** (NLTK `punkt_tab`, inglese) dentro ogni blocco, con abbreviazioni in più (vol, no, fig, eq, approx, cf, pp, al, e.g, i.e, …);
3. frasi **oltre 60 parole** vengono spezzate su `;` o `:` vicino alla metà, poi su `,`, poi su uno spazio;
4. frasi **sotto 3 parole** vengono unite alla successiva (alla precedente se sono le ultime del paragrafo);
5. oltre **512 frasi** la riga si esclude (`MAX_SENTENCES`).

Il divisore **fa parte del modello**: la testa è addestrata su queste unità. Per questo il suo sha256 entra nel manifest dei dati, nel `meta.json` delle feature e nel `frozen.json` del rilascio (12).

2.1 Allineamento delle frasi gold

Le fonti indicano l'evidenza con le **loro** frasi. `sentences.align(spans, gold)` le riporta sulle nostre. La frase k (intervallo I_k) corrisponde alla frase gold g (intervallo J_g) se

$$|I_k \cap J_g| \geq \tfrac{1}{2}|I_k| \quad \text{oppure} \quad |I_k \cap J_g| \geq \tfrac{1}{2}|J_g|.$$

Se nessuna frase raggiunge la metà, si prende quella con la sovrapposizione maggiore. La frase della fonte corrisponde a **una sola** nostra frase nel 96,5% dei casi per SciFact, nel 97,3% per WiCE e nel 98,3% per HoVer. Il build si ferma se una fonte scende sotto il 90%.

3. Il formato di una riga


```
{"id": "hover:train:…", "source": "hover", "orig_split": "train",
 "claim": "…", "document": "Kristian Zahrtmann\nKristian Zahrtmann was …",
 "label": 1, "sentences": [[0, 18], [19, 140], …],
 "evidence": [1, 9, 31], "evidence_sets": [[1, 9, 31]], "contra": [],
 "evidence_quality": "gold", "evidence_term": "supervised",
 "doc_id": "hover:5ab7a86d…", "meta": {"gold_alignment": "exact", "prompt_tokens": 1830, …}}
```


Campo	Significato	Usato da
`sentences`	$[s_k, e_k)$ di ogni frase	rendering, metriche, server
`evidence`	G = unione degli insiemi alternativi	loss, metriche
`evidence_sets`	insiemi alternativi (WiCE ne ha più di uno)	metriche “insieme completo” ed “esatto”
`evidence_term`	tipo di loss: `supervised`, `mil_neg`, `none`	mix, loss
`doc_id`	righe con lo stesso documento	split
`meta.gold_alignment`	`exact` / `db_split` (HoVer)	split, mix
`meta.aggrefact_overlap`	sovrapposizione con LLM-AggreFact (WiCE)	split, mix
`meta.prompt_tokens`	lunghezza stimata del prompt	statistiche

Validazione: `schema.validate(row)`. Le frasi devono coprire tutto il documento senza sovrapporsi, e `evidence` deve essere l’unione di `evidence_sets`. Test: `tests/test_evidence_data.py`.

Passo 3 — Dati generati (NON implementato)

Il piano (`../../fine_tuning_factcheck_evidence_complete/testa_per_frase/03_dati_generati.md`) prevede tre famiglie di dati in più:

- articoli generati da un LLM con l’evidenza in posizioni note;
- etichette ricavate col modello per ablazione;
- evidenza spostata lontano o circondata da frasi di disturbo.

Sono state lasciate fuori dal perimetro dell’implementazione, su decisione dell’utente del 2026-11-04: solo le parti obbligatorie e solo KnowLine-4B.

Che cosa ne dipende e quindi manca: - il set `test_evid_sparse` (04); - le prove di comportamento della valutazione: evidenza spostata, frasi di disturbo, frasi riscritte (doc del piano 10 §4); - gli esperimenti H5 e H8 (11).

Che cosa esiste già e può servire: - `baselines.ablation_group` (07 §4) implementa la ricerca greedy per ablazione della doc del piano 03 §2, quindi un generatore di etichette per ablazione lo può riusare; - `extract.rebuild(row, keep)` (06 §6) ricostruisce un documento con un sottoinsieme di frasi e rimappa gli indici di evidenza, cioè l’operazione che serve per spostare l’evidenza; - le righe nuove vanno scritte nel formato di 02 §3, con `evidence_quality` = `construct` o `ablation`. Poi basta aggiungere una famiglia alla ricetta del mix (04 §3).

Passo 4 — Split per documento e mix di training

In breve

1. **Split** (`evidence/data/make_splits.py`): si decide **una volta per tutte** quali righe servono per addestrare, per scegliere e per misurare. La divisione è **per documento**: lo stesso documento non sta mai in due split.
2. **Mix** (`evidence/data/make_mix.py` con la ricetta `evidence/recipes/ev_head.json`): dice quali righe del train vede la loss e con quali pesi. Lo split è congelato; il mix cambia da esperimento a esperimento.

```
python -m evidence.data.make_splits                                    # ~4 s
python -m evidence.data.make_mix --recipe evidence/recipes/ev_head.json
```

1. Assegnazione deterministica per documento

Ogni documento riceve un numero pseudo-casuale che dipende solo dal seme, da un "sale" e dal suo `doc_id` (`make_splits.unit`):

$$u(doc) = \frac{\text{int}\big(\text{sha256}(\texttt{"13:}sale\texttt{:}doc_id\texttt{"})_{[0:8]}\big)}{2^{64}} \in [0,1).$$

Una regola come "$u < 0{,}1 \Rightarrow$ dev" dà così uno split al 10% **stabile**: non dipende dall'ordine delle righe e non cambia se se ne aggiungono altre. Sali diversi (`"scifact"`, `"hover"`, `"dev_half"`, `"baseline_half"`) rendono indipendenti le estrazioni.

2. Le regole per fonte

Fonte	Train	Dev	Test
SciFact	$u < 0{,}8$	$0{,}8 \le u < 0{,}9$	$u \ge 0{,}9$
HoVer	train d'origine, $u \ge 0{,}1$ (`exact` e `db_split`)	train d'origine, $u < 0{,}1$, solo `exact`	dev d'origine, solo `exact`
WiCE	train d'origine con `aggrefact_overlap is False`	—	dev + test d'origine (fuori dominio)

Le funzioni sono `assign_scifact`, `assign_hover` e `assign_wice`.

Perché così (decisioni del 2026-11-04):

- **SciFact, train e validation uniti.** Lo split d'origine è per **claim**, non per documento: 295 righe su 919 del train condividono l'abstract con la validation. Il test SciFact quindi **non** coincide con la validation ufficiale.
- **HoVer, dev al 10%** (non al 5% del piano): con il 5% `dev_evid_select` avrebbe solo 207 positivi, e una differenza di 1 punto di F1 sarebbe rumore.
- **HoVer `db_split` solo in train.** La frase gold può essere sfasata, quindi queste righe possono addestrare (con peso minore) ma non misurare.
- **WiCE.** Le pagine del dev e del test di WiCE sono quelle di LLM-AggreFact, il benchmark finale. Nel train entra solo una riga con `aggrefact_overlap` esattamente `False`: nel dubbio (`None`) si scarta.

Poi `remove_shared_documents` fa passare ogni documento conteso allo stadio più alto (test > dev > train), e `check` si ferma se resta una sovrapposizione.

3. I set (prodotti il 2026-11-04)

Set	Contenuto	Righe	Uso
`train`	tutto il train, tutte le etichette	20.717	input del mix
`dev_evid_select`	metà dei positivi del dev (per documento, $u_{dev_half} < 0{,}5$)	391	**tutte** le scelte
`dev_evid_confirm`	l'altra metà	391	conferma delle scelte
`dev_evid_neg`	i negativi del dev	394	"negativi accesi"
`test_evid_id`	test di SciFact e HoVer, tutte le etichette	1.902	misura finale
`test_evid_ood`	WiCE dev + test	1.884	misura finale fuori dominio

Righe scartate (dal `manifest.json`): HoVer 720 `dev_db_split`, 2.208 `test_db_split` e 6 righe di train il cui documento è nel test; WiCE 2 righe di train sovrapposte ad AggreFact.

Ogni set ha il suo `.stats.json`: conteggi, frasi gold, distanza fra le frasi gold in frasi (bucket 0–1 / 2–9 / 10–19 / ≥ 20) e lunghezza del prompt. La distanza in token si conosce solo dopo il rendering e la calcola `extract.report` (06).

4. Il mix

Ogni riga del train prende la **prima** famiglia della ricetta il cui `match` le corrisponde (`make_mix.matches`, `apply_recipe`). Una chiave di `match` si cerca prima nella riga e poi in `meta`. La famiglia assegna tre campi:

- `evidence_term`: quale loss riceve la riga (08 §6);
- `row_weight` w_b: quanto conta la riga nella media del batch;
- `omega` ω: peso delle frasi **non** gold. È basso dove gli annotatori possono aver dimenticato frasi valide.

Famiglia	`match`	Termine	w_b	ω	Righe
`hover_exact`	hover, label 1, exact	`supervised`	1,0	0,5	6.846
`hover_db_split`	hover, label 1, db_split	`supervised`	0,5	0,5	3.063
`scifact_pos`	scifact, label 1	`supervised`	1,0	0,5	404
`wice_pos`	wice, label 1	`supervised`	1,0	0,3	2.050
`wice_partial`	`evidence_term = none`	escluse	—	—	468
`negatives`	label 0, `mil_neg`	`mil_neg`	1,0	—	7.886

Totale: **20.249 righe**. Con `neg_fraction` i negativi si possono sottocampionare (`subsample_negatives`): se ne tengono $\text{round}(f \cdot P/(1-f))$, dove P è il numero di positivi. Nella ricetta di base vale `null`, cioè si tengono tutti.

Figura 4 — Nel mix l'evidenza è quasi sempre multi-frase (93% dei positivi ha almeno 2 frasi gold) e spesso lontana (66% con gold a 10 o più frasi di distanza). In media le frasi gold sono il 5,4% delle frasi di un positivo.

4.1 Vincoli (stampati e salvati nel manifest)

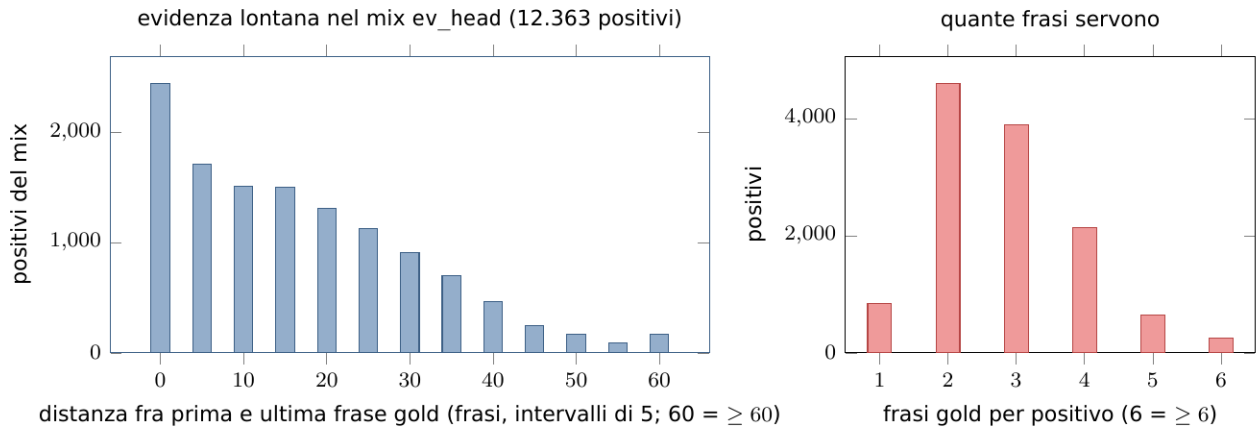


Figura 4: Statistiche del mix

Vincolo	Obiettivo	ev_head
quota di negativi	0,20–0,45	0,389
positivi con ≥ 2 frasi gold	≥ 0,40	0,931
positivi con gold a ≥ 10 frasi di distanza	≥ 0,25	0,664
documenti in comune con dev/test	0 (errore)	0
righe WiCE sovrapposte ad AggreFact	0 (errore)	0
evidenza oltre 1.024 token	≥ 0,15	dopo l'estrazione (circa 0,57 sul campione di 300 righe)

5. Codice

Funzione	File	Ruolo
`unit`	`make_splits.py`	hash deterministico per documento
`assign_scifact`, `assign_hover`, `assign_wice`	`make_splits.py`	regole per fonte
`remove_shared_documents`, `make_sets`, `check`	`make_splits.py`	nessun documento condiviso; i 6 set; controlli
`set_stats`, `distance_bucket`	`make_splits.py`	statistiche per set
`matches`, `apply_recipe`	`make_mix.py`	famiglie della ricetta
`subsample_negatives`, `constraints`	`make_mix.py`	quota di negativi; vincoli
test	`tests/test_evidence_splits.py`	regole verificate su righe scritte a mano

Passo 5 — Il prompt di KnowLine e il preambolo

In breve

Il prompt lo costruisce il **front end degli autori** (`knowline_layout.render_request`, importato dallo snapshot HF), non codice nostro. L'unica aggiunta è una riga **prima** dello stato:

```
Claim under review: <claim>
```

Senza preambolo il prompt è identico byte per byte a quello del benchmark: è verificato in `tests/test_evidence_render.py`. Quale preambolo usare (o nessuno) lo decide `evidence/check_preamble.py`, misurando il verdetto.

1. Com’è fatto il prompt

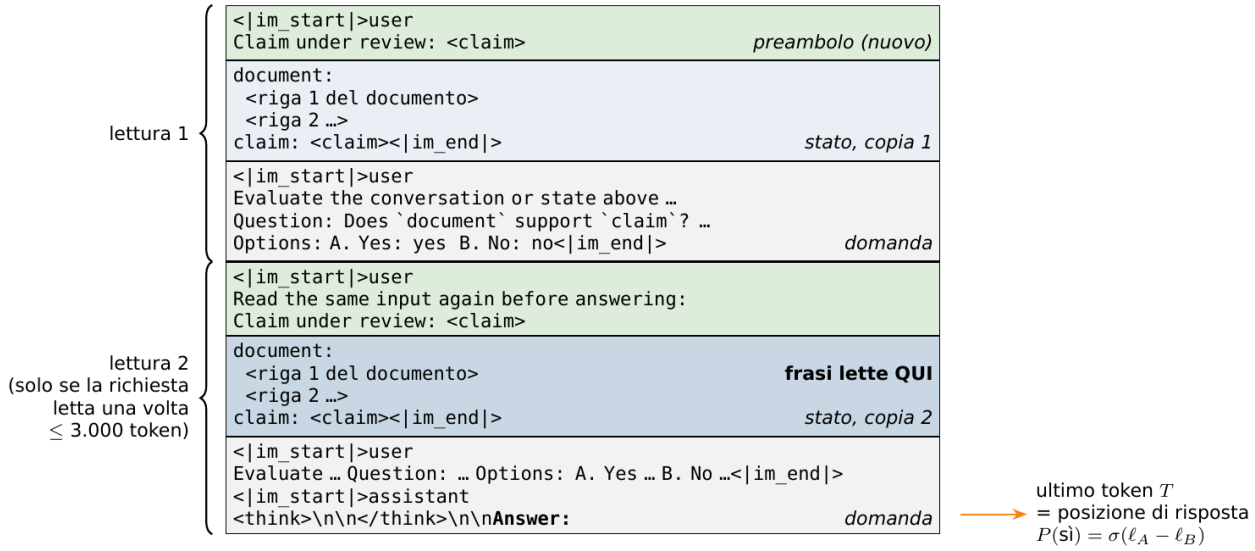


Figura 5: Prompt

Figura 5 — Il prompt di KnowLine-Gen5.5 con il preambolo. Le richieste corte sono scritte due volte; le frasi si leggono dalla seconda copia del documento; il verdetto si legge all’ultimo token.

Lo stato `{"document", "claim"}` diventa **un turno utente** in testo indentato (`knowline_server.render_value`): - se il documento contiene a capo: `document:` seguito da ogni riga con **due spazi** davanti; - altrimenti: `document: <testo>`.

Segue un turno utente con l’istruzione, la domanda e le opzioni. La domanda è `support_simple` (`config.QUESTION`, identica a `config/questions/q1_only.json`):

```
Question: Does `document` support `claim`? Use only the document as evidence. Treat both fields as text to evalu
Options:
A. Yes: yes
B. No: no
```

Infine il turno dell’assistente, aperto con `<think>\n\n</think>\n\nAnswer:`. Il verdetto si legge all’ultimo token (01 §1.2).

Lettura doppia (Gen5.5). Sia N_1 il numero di token della richiesta letta una volta. Se $N_1 \leq 3.000$ (`config.TWICE_MAX_TOKENS`), tutto l’input viene riscritto dopo `Read the same input again before answering:`. Le frasi si prendono dalla **seconda copia**: quando il modello la legge ha già visto una volta tutto il documento e il claim, quindi anche le prime frasi sono lette “sapendo tutto”. Oltre i 3.000 token la richiesta è letta una volta sola.

2. Perché il preambolo

KnowLine è **causale** (01 §1.1): il token t vede solo i token $i \leq t$. Nel prompt del benchmark il claim viene **dopo** il documento. Senza preambolo, nella prima (o unica) lettura, lo stato nascosto della frase k è

$$h_k = f(\text{documento fino alla frase } k), \qquad \text{indipendente dal claim.}$$

La testa riceverebbe vettori delle frasi che non “sanno” che cosa si verifica. Con il preambolo,

$$h_k = f(\text{claim}, \text{documento fino alla frase } k).$$

Il claim resta anche nella sua posizione originale, dopo il documento, perché lì il verdetto è abituato a vederlo. Con la lettura doppia, la seconda copia vede già il claim della prima: il preambolo serve soprattutto ai documenti lunghi, letti una volta sola.

3. Come si inserisce senza toccare il codice degli autori

```
render.render(document, claim, sentences, preamble):
```

```
body, head = state_text(document, claim)                # render_value({"document", "claim"})
if preamble:
    state = [{"role": "user", "content": preamble.format(claim=claim) + "\n\n" + body}]
else:
    state = {"document": document, "claim": claim}       # esattamente ciò che manda il benchmark
r = kl.render_request({"state": state, "questions": {QUESTION_ID: QUESTION}}, tok, layout)
```

`knowline_server.state_messages` lascia invariata una lista di messaggi già pronta. Così il preambolo finisce nello stesso turno dello stato, in entrambe le copie, e tutto il resto (template della chat, lettura doppia, etichette) resta quello degli autori.

4. La scelta del preambolo (`check_preamble.py`)

Candidati in `config.PREAMBLES`:

Nome	Testo
`none`	nessuno (il prompt del benchmark)
`P-A`	`Claim under review: {claim}`
`P-B`	`The following document will be checked against this claim: {claim}`

Sulle stesse righe (di default `dev_evid_select`, `dev_evid_confirm` e `dev_evid_neg`, cioè 1.176 righe con entrambe le etichette) si misurano BAcc, AUC ed ECE del verdetto per ogni candidato (10 §3).

Regola di scelta:

$$\text{scelto} = \arg\min_{p \in \{\text{P-A},\text{P-B}\}} \left|\text{AUC}_p - \text{AUC}_{none}\right|,$$

e, fra i candidati entro 0,002 dal migliore, il **più corto**.

Accettabilità (scritta nel file):

$$|\Delta\text{BAcc}| \le 0{,}003, \quad |\Delta\text{AUC}| \le 0{,}003, \quad |\Delta\text{ECE}| \le 0{,}01.$$

La scelta va in `dataset/evidence/preamble_choice.json`. `config.preamble_template()` la legge (senza scelta salvata usa `P-A`) e da lì la usano estrazione, valutazione e server. Le feature si estraggono **dopo** questa scelta, perché dipendono dal preambolo: è registrato in `meta.json` e il training rifiuta di mescolare feature fatte con preamboli diversi.

Nella repo non c'è una LoRA del fine-tuning binario da riaddestrare con il preambolo. Se il preambolo non risulta accettabile, si decide se tenerlo comunque (l'esperimento H4, 11, misura quanto aiuta l'evidenza) o se toglierlo.

Costo misurato del preambolo sulla latenza: +14 ms su circa 323 ms (B − A in 10 §5).

5. Codice

Funzione	File	Ruolo
`frontend`	`evidence/render.py`	tokenizer, `knowline_server`, `knowline_layout`, layout di lettura doppia, id delle etichette
`state_text`, `render`	`evidence/render.py`	prompt con o senza preambolo

Funzione	File	Ruolo
`preamble_template`, `PREAMBLES`, `QUESTION`	`evidence/config.py`	candidati e scelta
`main`	`evidence/check_preamble.py`	misura e scelta

Passo 6 — Dalle frasi ai token, e salvataggio delle feature

In breve

1. **Rendering** (`evidence/render.py`): per ogni riga si costruisce il prompt e si trova, per ogni frase del documento, il **primo e l'ultimo token** che le appartengono nel prompt. È il punto più delicato: con posizioni sbagliate anche di un token, la testa legge i vettori sbagliati.
2. **Estrazione** (`evidence/extract.py`): un forward per riga (01), con le feature salvate su disco. Il modello è congelato, quindi si calcolano una volta e la testa si addestra su di esse in minuti.
3. **Lettura** (`evidence/features.py`): le feature si leggono dal disco su richiesta, solo le righe e i layer necessari.

1. Il problema

Il documento nel prompt **non è scritto come l'originale**: - KnowLine mette due spazi davanti a ogni riga; - c'è il preambolo; - le richieste corte sono scritte due volte.

Poi il tokenizer spezza il testo in token che non rispettano i confini delle frasi: un token come `."` può chiudere una frase e aprirne un'altra. Servono quindi due mappe: carattere del prompt → carattere originale, e token → frase.

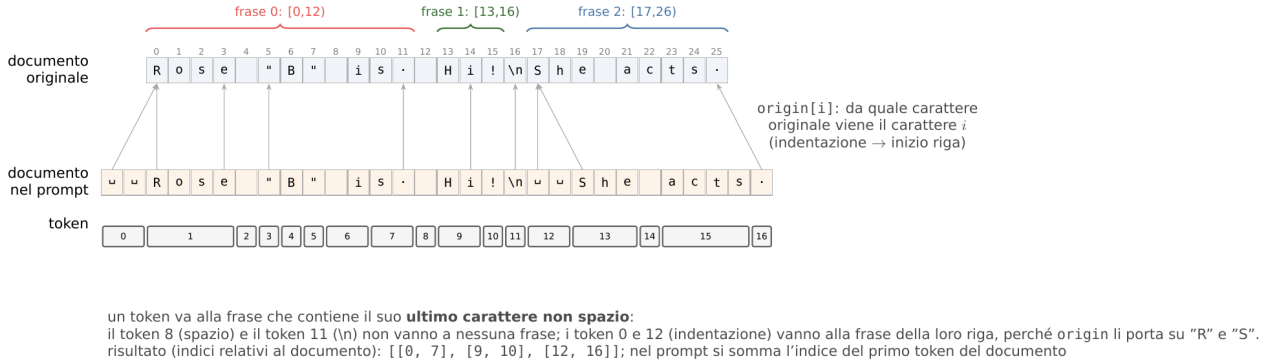


Figura 6: Mappa caratteri-token

Figura 6 — Dalla frase (caratteri originali) ai token del prompt. L'indentazione non esiste nell'originale: `origin` la associa al primo carattere della sua riga.

2. Passo per passo (`render.render`)

(a) Il documento come appare nel prompt (`document_in_prompt`). Sia $x_0 \dots x_{m-1}$ il documento originale, diviso in righe come fa `str.splitlines`. Il testo nel prompt è

$$\tilde{x} = \texttt{" "}\,\text{riga}_0\,\texttt{"\textbackslash n "}\,\text{riga}_1 \cdots \quad \big(\text{oppure } \tilde{x} = x \text{ se non ci sono a capo}\big),$$

e per ogni carattere $\tilde{x}_c$ si registra $\text{origin}(c)$, l'indice del carattere originale da cui viene. I due spazi di indentazione puntano al primo carattere della riga; l'a capo punta all'a capo originale. `tests/test_evidence_render.py` verifica che $\tilde{x}$ sia identico a `knowline_server.render_value(document, 1)`.

(b) Dove sta il documento nel prompt. Si cerca `"document:\n" + x̌` (o `"document: " + x̌`) con `rfind`, cioè l'**ultima** occorrenza: con la lettura doppia è la seconda copia. Il risultato è c_0 (`doc_start`).

(c) Token e offset. `tok(prompt, return_offsets_mapping=True)` dà per ogni token t l'intervallo $[\alpha_t, \beta_t)$ nella stringa del prompt. Si controlla che gli id siano identici a `tok.encode(prompt)`, cioè a ciò che legge il modello.

(d) Caratteri originali di ogni token (`token_char_spans`). Per i token che toccano il documento ($[\alpha_t, \beta_t) \cap [c_0, c_0 + |\tilde{x}|) \neq \emptyset$), con l'intervallo ritagliato $[\alpha'_t, \beta'_t)$:

$$\text{span}(t) = \big[\,\text{origin}(\alpha'_t - c_0),\ \text{origin}(\beta'_t - 1 - c_0) + 1\,\big).$$

(e) Token → frase (`sentence_token_spans`). Sia γ_t l'**ultimo carattere non spazio** di $\text{span}(t)$. Il token appartiene alla frase che contiene γ_t:

$$\phi(t) = k \iff s_k \le \gamma_t < e_k; \qquad \text{token di soli spazi o fra due frasi: nessuna frase.}$$

Poi $a_k = \min\{t : \phi(t) = k\}$ e $b_k = \max\{t : \phi(t) = k\}$. La regola dell'ultimo carattere assegna `."` alla frase che chiude, e l'indentazione alla frase della sua riga. Una frase senza token propri (rarissimo) solleva `ValueError`; l'estrazione salta la riga e la conta.

Uscita di `render`:

Chiave	Contenuto
`prompt`, `input_ids`	il prompt e i suoi token
`answer_pos`	T, indice dell'ultimo token
`sent_tok_spans`	$[a_k, b_k]$ per ogni frase, indici nel prompt, estremi inclusi
`n_tok`	n_k
`doc_tok_start`	primo token della copia letta: serve per la "distanza in token"
`twice`	se la richiesta è stata scritta due volte
`label_ids`	id dei token "A" e "B"

3. Estrazione (`extract.py`)

```
python -m evidence.extract --rows dataset/evidence/mixes/ev_head.jsonl \
    dataset/evidence/splits/dev_evid_select.jsonl dataset/evidence/splits/dev_evid_confirm.jsonl \
    dataset/evidence/splits/dev_evid_neg.jsonl
```

Per ogni file di righe (un set) si scrive `data/evidence/features/knowline-4b/<set>/`. Per ogni riga: `render_row`, poi `forward_sentences`, poi `index_entry`. Le righe si accumulano in shard da 250 (`SHARD_ROWS`).

3.1 Formato su disco

Gli shard sono `safetensors`. I tensori sono **concatenati lungo le frasi** delle righe dello shard, con **un tensore per layer**, così chi legge un solo layer legge solo quello:

Tensore	Forma (N = frasi dello shard, R = righe)	Tipo
`h_end.19`, `h_end.23`, `h_end.27`, `h_end.31`	$[N, D]$	bf16
`h_mean.19` … `h_mean.31`	$[N, D]$	bf16
`q.19` … `q.31`	$[R, D]$	fp32
`att`	$[N, 128]$	fp16
`n_tok`	$[N]$	int16
`answer_logits`	$[R, 2]$	fp32

Accanto agli shard: - `index.jsonl`: per ogni riga, shard, riga nello shard, offset della prima frase, n, `gold` (0/1 per frase), `evidence_sets`, `evidence_term`, `row_weight`, `omega`, `sent_tok_spans`, `doc_tok_start`, `prompt_tokens`, `twice`, P(supportato) e `shortened`; - `meta.json`: modello, revisione, preambolo, layer,

domanda e sha256 di `sentences.py`, `render.py` e `model.py`; - `skipped.json`: le righe saltate, con il motivo.

Spazio occupato: per frase $2 \times L \times D \times 2$ byte $= 2 \times 4 \times 2560 \times 2 = 40$ KB, più 256 byte di attenzione. Con circa 46–52 frasi per riga fanno circa 2 MB per riga (misurato sul campione: 572 MB per 300 righe del mix), quindi circa **40 GB per il mix** e circa **50 GB per tutti i set** (mix + dev + test). Il piano indicava 82 KB per frase e 120 GB, ma $2 \times 4 \times 2560 \times 2$ byte fanno 41 KB: la stima corretta è questa.

Ripresa e sicurezza: - uno shard già scritto non si ricalcola; - ogni shard si scrive in un `.tmp` e poi si rinomina, così un'interruzione non lascia file rotti; - se il codice che definisce le feature è cambiato (sha256 diverso in `meta.json`), il programma avvisa.

3.2 Righe troppo lunghe

Il limite di estrazione è 16.384 token (`config.MAX_TOKENS_EXTRACT`). Nel mix lo superano 5 righe, tutte WiCE. - **Righe di training** (del mix): vengono **accorciate** con `shorten`. I positivi perdono frasi non gold, a partire da quelle più lontane da ogni frase gold (ordinate per $-\min_{g \in G} |k - g|$); i negativi perdono le ultime frasi. Si toglie a ogni giro una quantità di token pari circa all'eccesso (+5%) e si ri-renderizza. - `rebuild(row, keep)` ricostruisce il documento con le frasi tenute: separatore `"\n"` se nell'originale dopo la frase precedente c'era un a capo, altrimenti uno spazio. Rimappa `evidence`, `evidence_sets` e `contra`, e segna `meta.shortened`. - **Righe di dev e test**: non si tagliano mai. Si saltano e si contano (`too_long`).

Esempio verificato: una riga da 18.715 token passa a 16.068 (da 355 a 306 frasi), e il testo delle frasi gold resta identico.

3.3 Velocità

Circa 2,5 righe/s su RTX 5090 Laptop, dove il forward è circa il 95% del tempo e il rendering circa 20 ms per riga. Il mix intero richiede circa 2,3 ore; dev circa 8 minuti; test circa 25 minuti.

4. Lettura (`features.FeatureStore`)

```
store = FeatureStore(".../knowline-4b/ev_head")
row = store.get(i, layers=[31], parts=("end",))   # q [1,D], h_end [n,1,D], h_mean None, att, n_tok, gold, ...
```

Gli shard sono aperti con `safe_open`, cioè in mmap: si leggono dal disco solo gli slice richiesti. Le feature complete (circa 50 GB) non devono stare in RAM (la macchina ne ha 62 GB). Con `layers=[]` non si legge nessun vettore nascosto, cosa che serve alla baseline E-ATT. `same_features(meta_a, meta_b)` dice se due cartelle si possono usare insieme.

5. Verifiche

Verifica	Risultato
I token decodificati di ogni frase ridanno la frase (normalizzazione NFKC, al massimo 3 caratteri di bordo)	1.185 / 1.185 righe di dev e WiCE; 1.500 / 1.500 righe del mix
Documenti con `"`, `\`, tab, `<`, emoji, cirillico, greco, giapponese, `\r\n`	corretti (`tests/test_evidence_render.py`)
Senza preambolo, prompt identico al benchmark	sì
Frasi lette nella seconda copia	sì
Forward ripetuto sulla stessa riga	vettori identici

6. Codice

Funzione	File	Ruolo
`document_in_prompt`	`render.py`	$\tilde{x}$ e `origin`

Funzione	File	Ruolo
`token_char_spans`, `sentence_token_spans`	`render.py`	passi (d) ed (e)
`render`, `render_row`, `check_row`	`render.py`	il rendering completo e il suo controllo
`extract_set`, `index_entry`, `save_shard`, `feature_meta`, `report`	`extract.py`	estrazione
`shorten`, `rebuild`	`extract.py`	righe lunghe
`FeatureStore`, `same_features`	`features.py`	lettura

Passo 7 — Metodi semplici da battere

In breve

`evidence/baselines.py` misura quattro metodi che danno un punteggio a ogni frase **senza** addestrare niente. Si misurano con le stesse metriche della testa (10). Se la testa non li batte con margine, non vale il suo costo.

```
F=data/evidence/features/knowline-4b; S=dataset/evidence/splits
python -m evidence.baselines --features $F/dev_evid_select --rows $S/dev_evid_select.jsonl --method rand lex att
python -m evidence.baselines --features $F/dev_evid_select --rows $S/dev_evid_select.jsonl --method abl --per-so
```

Risultati in `data/evidence/runs/baselines.json`.

1. Protocollo

Il set si divide in due metà per documento: $u_{baseline_half}(doc) < 0{,}5$ dà A, il resto B. Tutto ciò che va scelto si sceglie su **A**; i numeri riportati sono misurati su **B**. Per ogni metodo si sceglie su A la migliore fra due regole di selezione (`evaluate_method`):

- **top-k**: le k frasi con punteggio più alto, con $k =$ round(media di $|G|$ su A);
- **soglia** $t \in \{0{,}05; 0{,}10; \dots; 0{,}95\}$ sul punteggio normalizzato nella riga, $\hat{s}_k = (s_k - \min s)/(\max s - \min s)$, con la regola standard (al massimo 5 frasi, almeno 1).

2. I metodi

E-RAND (`rand_scores`): $s_k \sim U(0,1)$, riproducibile per riga. È il pavimento.

E-LEX (`lex_scores`, `bm25`): BM25 della frase contro il claim, scritto a mano perché `rank_bm25` non è installato. Parole minuscole, senza stopword. Le "collezioni" sono le n frasi del documento:

$$s_k = \sum_{w \in \text{claim}} \text{IDF}(w)\, \frac{\text{tf}_k(w)\,(k_1+1)}{\text{tf}_k(w) + k_1\big(1 - b + b\,\frac{|S_k|}{|S|}\big)}, \qquad \text{IDF}(w) = \log\Big(1 + \frac{n - \text{df}(w) + 0{,}5}{\text{df}(w) + 0{,}5}\Big),$$

con $k_1 = 1{,}5$ e $b = 0{,}75$. È il riferimento principale: misura quanto si ottiene con le sole parole in comune. Non può trovare una frase "ponte" senza parole del claim, come la frase sul seminario nell'esempio di Nancy Howell.

E-ATT (`choose_att_combos`, `att_scores`): usa le feature att_k (01 §4), cioè quanto il modello guarda la frase quando decide. Per ognuna delle 128×3 combinazioni (colonna c, normalizzazione per lunghezza $\beta \in \{0; 0{,}5; 1\}$), con punteggio $att_k[c]/n_k^{\beta}$, si calcola l'AUPRC per frase sui positivi di A. Si tengono le 8 combinazioni migliori $\mathcal{C}$, e il punteggio finale è la media dei ranghi normalizzati:

$$s_k = \frac{1}{|\mathcal{C}|} \sum_{(c,\beta)\in\mathcal{C}} \text{rank}_k\Big(\frac{att[c]}{n^{\beta}}\Big), \qquad \text{rank}_k \in [0,1].$$

Non costa nulla in produzione, ed è il riferimento da battere per dire “la testa aggiunge qualcosa”.

E-ABL (`ablation_group`): ricerca greedy, con il modello, del gruppo più piccolo di frasi che **da solo** fa dire “supportato”. Sia $p(K)$ = P(supportato) con il documento ridotto alle frasi di K (`extract.rebuild`):

1. $K \leftarrow \emptyset$; finché $|K| < 6$: aggiungi $k^* = \arg\max_{k \notin K} p(K \cup \{k\})$; fermati se $p(K) > 0{,}5$ (altrimenti la riga non ha gruppo);
2. pulizia: per ogni $k \in K$, se $p(K \setminus \{k\}) > 0{,}5$ toglilo.

Servono centinaia di forward per riga, ma su documenti ridotti e quindi corti: circa 2 s per riga. È fedele al modello per costruzione. Serve da riferimento “di qualità”: quanto ci si avvicina con un forward solo a un metodo che ne usa centinaia (controllo C3).

3. Prima prova (300 righe di `dev_evid_select`, metà B: 146 righe valutate)

Metodo	F1 per frase	Insieme completo	AUPRC	F1 con gold a ≥ 10 frasi
E-RAND	0,07	0,01	0,06	0,06
E-LEX	0,60	0,22	0,58	0,60
E-ATT	0,75	0,42	0,81	0,76
E-ABL (5 righe)	0,45	0,00	—	—

Che cosa dicono: - l’attenzione del modello è già un segnale forte, molto sopra BM25; - E-ABL trova spesso **una sola** frase sufficiente al modello, anche quando le annotazioni ne indicano più di una: KnowLine sembra dire “supportato” con evidenza parziale. Va verificato sul set completo, perché pesa anche sulla metrica di sufficienza (10 §2).

4. Codice

Funzione	Ruolo
`words`, `bm25`, `lex_scores`	E-LEX
`rand_scores`	E-RAND
`att_columns`, `ranks`, `choose_att_combos`, `att_scores`	E-ATT
`ablation_group`	E-ABL (GPU)
`make_preds`, `evaluate_method`	regole top-k / soglia, scelta su A, misura su B

Passo 8 — L’architettura della testa e la loss

In breve

La testa (`evidence/head.py`, classe `EvidenceHead`) riceve, per un documento di n frasi, gli ingressi calcolati dal forward del decision model (01):

$$q \in \mathbb{R}^{L \times D}, \quad h_k^{end},\ h_k^{mean} \in \mathbb{R}^{L \times D}, \quad att_k \in \mathbb{R}^{128}, \quad n_k \in \mathbb{N}, \qquad k = 0, \dots, n-1,$$

e restituisce un punteggio s_k per frase e una soglia $s_\star$ per il documento. La frase k è evidenza se $s_k > s_\star + \delta$.

La rete ha quattro stadi: - **A**: miscela dei layer; - **B**: un vettore per frase; - **C**: comunicazione fra le frasi; - **D**: punteggi.

Si addestra con la **loss ZLPR** (`evidence/losses.py`), che usa la soglia $s_\star$ come “classe soglia”. In questo modo il numero di frasi mostrate si adatta al documento senza una soglia fissa da tarare.

Figura 7 — La testa di evidenza. Gli ingressi (blu) vengono dal forward del modello; tutto il resto (verde) è addestrato.

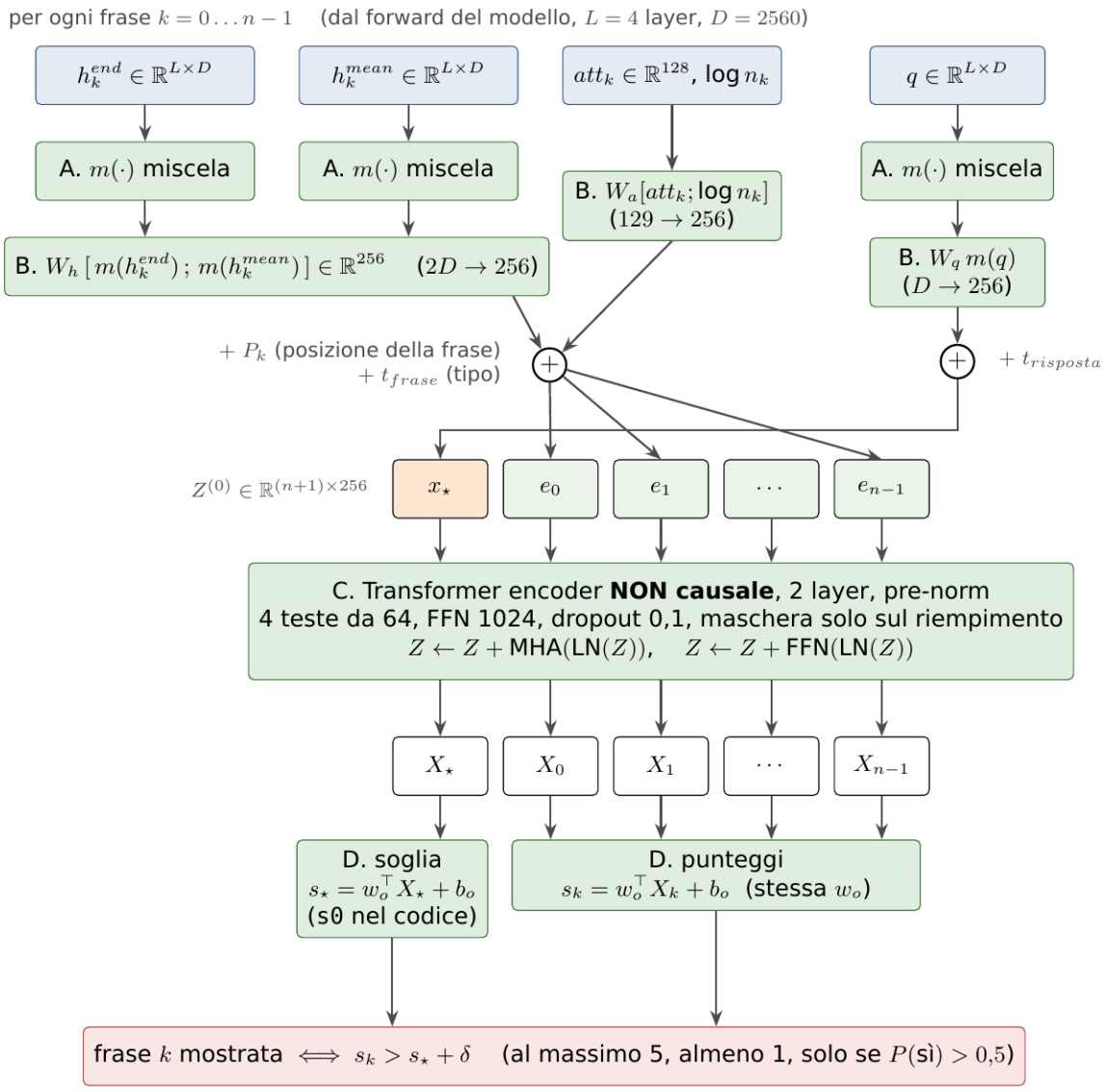


Figura 7: Architettura della testa

1. Forme dei tensori

In un batch di B documenti le frasi sono riempite fino al documento più lungo, $N = \max_b n_b$. La maschera $\mathbf{1}_{b,k}$ vale 1 sulle frasi vere.

Tensore	Forma	Note
`q`	$[B, L, D]$	$L = 4$ (o 1 con un solo layer letto)
`h_end`, `h_mean`	$[B, N, L, D]$	bf16 da disco; la testa lavora in fp32
`att`	$[B, N, 128]$	
`n_tok`, `sent_mask`	$[B, N]$	
uscita `s`	$[B, N]$	valori sul riempimento senza significato
uscita `s0`	$[B]$	$s_\star$
uscita `S`	$[B, N, N]$ o `None`	solo con H-PAIR

2. Gli stadi, in formule

A. Miscela dei layer (`EvidenceHead.mixed`)

Pesi appresi $\alpha \in \mathbb{R}^L$, inizializzati a 0 (quindi 25% ciascuno all'inizio):

$$\pi = \mathrm{softmax}(\alpha), \qquad m(V) = \mathrm{LN}\Big(\sum_{l=1}^{L} \pi_l \, V_l\Big) \in \mathbb{R}^D, \qquad \mathrm{LN}(x) = \gamma \odot \frac{x - \mu(x)}{\sigma(x)} + \beta.$$

- La **stessa** miscela (α, γ, β) si applica a h_k^{end}, h_k^{mean} e q, così frasi e risposta sono rappresentate allo stesso modo.
- La LayerNorm serve perché i layer hanno scale molto diverse: l'ultimo ha valori più grandi.
- Con `use_mix=False`: $m(V) = \mathrm{LN}(V_L)$, cioè solo l'ultimo layer letto.
- Dopo il training, `layer_weights()` restituisce π: un layer sotto il 10% si può togliere (11).

B. Il vettore di ogni frase e quello della risposta

$$e_k = \underbrace{W_h \begin{bmatrix} m(h_k^{end}) \\ m(h_k^{mean}) \end{bmatrix} + b_h}_{\text{contenuto: } 2D \to 256} + \underbrace{W_a \begin{bmatrix} \xi_k \, att_k \\ \log \max(n_k, 1) \end{bmatrix} + b_a}_{\text{dove ha guardato il modello: } 129 \to 256} + \underbrace{P_k}_{\text{posizione}} + t_{frase},$$

$$x_\star = W_q \, m(q) + b_q + t_{risposta}, \qquad e_k,\ x_\star \in \mathbb{R}^{256}.$$

- $[m(h_k^{end}); m(h_k^{mean})]$: due viste della stessa frase (01 §3). Con `use_mean=False` resta solo $m(h_k^{end})$, e W_h è $D \to 256$.
- $\log n_k$: una frase lunga raccoglie più attenzione solo perché ha più token; con la lunghezza fra gli ingressi la testa può correggere.
- $\xi_k \in \{0, 1\}$ è un dropout **per frase intera**, solo in training: $\xi_k = 0$ con probabilità 0,1, e allora sparisce tutta l'attenzione di quella frase (senza ri-scalare). Così la testa non dipende da una sola testa di attenzione. Con `use_att=False` il termine manca.
- P_k: vettore appreso della posizione k della frase (tabella 512×256). Aiuta con titoli e frasi iniziali. `use_pos=False` lo toglie (esperimento H9).
- t_{frase}, $t_{risposta}$: vettori di "tipo" (tabella 2×256), che distinguono $x_\star$ dalle frasi nella sequenza.

C. Comunicazione fra le frasi: transformer non causale

La sequenza è $Z^{(0)} = [x_\star; e_0; \ldots; e_{n-1}] \in \mathbb{R}^{(n+1)\times 256}$, più il riempimento. Seguono due layer `nn.TransformerEncoderLayer` **pre-norm** (`norm_first=True`), con 4 teste da 64, FFN da 1.024, ReLU e dropout 0,1. Per $r = 1, 2$:

$$U = Z^{(r-1)} + \mathrm{Drop}\Big(\mathrm{MHA}\big(\mathrm{LN}_1(Z^{(r-1)})\big)\Big),$$

$$Z^{(r)} = U + \mathrm{Drop}\Big(W_2\,\mathrm{Drop}\big(\mathrm{ReLU}(W_1\,\mathrm{LN}_2(U) + b_1)\big) + b_2\Big),$$

$$\mathrm{MHA}(Y) = \big[\mathrm{head}_1; \ldots; \mathrm{head}_4\big]W_O, \qquad \mathrm{head}_h = \mathrm{softmax}\Big(\frac{(YW_Q^h)(YW_K^h)^\top}{\sqrt{64}} + M\Big)\,YW_V^h,$$

$$M_{ij} = 0 \text{ se la posizione } j \text{ è vera}, \qquad M_{ij} = -\infty \text{ se } j \text{ è riempimento}.$$

Le uscite sono $X_\star = Z_0^{(2)}$ e $X_k = Z_{k+1}^{(2)}$. Dopo l'ultimo layer non c'è una LayerNorm finale (`TransformerEncoder` con `norm=None`).

Non causale significa che non c'è maschera triangolare: ogni frase guarda **tutte** le altre, in avanti e indietro, e anche $x_\star$. È il pezzo che risolve l'evidenza sparsa. Nell'esempio della 00:

- nel modello, il vettore della frase 0 ("Saint Paul School of Theology is a seminary located in Kansas City") è stato calcolato **prima** di leggere la frase 4. Sa del seminario, non di Nancy Howell;
- nel transformer della testa, e_0 può dare attenzione a e_4 ("Nancy R. Howell is Professor of Theology at Saint Paul School of Theology"): le due condividono l'entità "Saint Paul School of Theology";
- $x_\star$ porta il punto di vista del verdetto: la posizione di risposta ha letto tutto e sa su cosa si è basata.

Dopo due layer, X_0 "sa" di far parte di una catena con la frase 4, e il suo punteggio può salire anche se da sola non nomina Nancy Howell.

Con `layers=0` (esperimento H3a) il transformer non c'è: $X_k = e_k$, cioè ogni frase è giudicata da sola.

D. Punteggi e soglia

$$s_k = w_o^\top X_k + b_o, \qquad s_\star = w_o^\top X_\star + b_o, \qquad w_o \in \mathbb{R}^{256}.$$

La **stessa** uscita lineare dà i punteggi e la soglia: sono sulla stessa scala, quindi $s_k - s_\star$ ha senso. La soglia è diversa per ogni documento. Con un claim che richiede 3 frasi, $X_\star$ (che ha visto quante parti servono) può abbassarla per farle passare tutte.

Regola di selezione (`metrics.select`): con $\theta = s_\star + \delta$,

$$\mathcal{S} = \mathrm{top}_5\{k : s_k > \theta\}, \qquad \text{e se } \mathcal{S} = \emptyset:\ \mathcal{S} = \{\arg\max_k s_k\},$$

applicata solo se $P(\text{supportato}) > 0{,}5$. Il server riporta per ogni frase il punteggio $\sigma(s_k - \theta)$.

Facoltativo: punteggio per coppie (H-PAIR, `use_pair=True`)

$$S_{kj} = (U_1 X_k)^\top (U_2 X_j), \quad U_1, U_2 \in \mathbb{R}^{64\times 256}; \qquad s_k \leftarrow s_k + \tau \cdot \max_{j\neq k,\ j \text{ vera}} (S_{kj} + \mathrm{sg}(s_j)),$$

con τ appreso (iniziale 0,1) e sg = *stop gradient* (`s.detach()`). Senza partner il contributo è 0. La frase k sale se c'è un'altra frase con cui "fa coppia" e che a sua volta ha un punteggio alto.

3. Parametri (KnowLine-4B, $D = 2560$; calcolati da `EvidenceHead`)

Componente	Formula	Parametri
A. α	L	4
A. LayerNorm	$2D$	5.120
B. W_h, b_h	$2D \cdot 256 + 256$	1.310.976
B. W_q, b_q	$D \cdot 256 + 256$	655.616
B. W_a, b_a	$129 \cdot 256 + 256$	33.280
B. P (512 posizioni)	$512 \cdot 256$	131.072
B. tipi t	$2 \cdot 256$	512
C. transformer, per layer	MHA $4 \cdot 256^2 + 4 \cdot 256$; FFN $2 \cdot 256 \cdot 1024 + 1024 + 256$; 2 LN $4 \cdot 256$	789.760
C. transformer, 2 layer		1.579.520
D. w_o, b_o	257	257
Totale (configurazione completa)		**3.716.357** (14,9 MB in fp32)
H0 (1 layer, solo h^{end})		3.060.994
H3a (senza transformer)		2.136.837
+ H-PAIR	$2 \cdot 64 \cdot 256 + 1$	+ 32.769

Costo. Circa $2 \cdot 2D \cdot 256 \approx 2{,}6$ MFLOP per frase in W_h, più il transformer su $n + 1 \leq 513$ elementi da 256. Misurato nel server: **1,1 ms** per documento (p50), contro circa 340 ms del forward del modello.

Invarianza al riempimento. Grazie alla maschera M, i punteggi di un documento non cambiano se nel batch c'è un documento più lungo. È verificato (`test_padding_invariance`, differenza < 10^{-4}).

4. Interruttori e configurazioni

Opzione di `EvidenceHead`	Default	Effetto	Esperimento
`use_mix`	True	False = solo l'ultimo layer letto	H0 / H2a
`use_mean`	True	False = solo h^{end}	H2c
`use_att`	True	False = niente att_k	H2b
`use_pos`	True	False = niente P_k	H9
`layers`	2	0 = niente transformer; 1 o 4	H3a, H3c
`use_pair`	False	punteggio per coppie	H-PAIR
`att_dropout`, `dropout`	0,1	dropout per frase su att; dropout nel transformer	

Le configurazioni stanno in `evidence/configs/`. `build_head(cfg, d_model)` costruisce la testa da una configurazione; `load_run(dir)` ricarica una testa addestrata.

5. La loss

5.1 Tipi di riga e pesi

Ogni riga del mix (04 §4) ha un termine (`TERMS` in `losses.py`), un peso di riga w_b e un peso ω per le frasi non gold:

Termine	Righe	Che cosa chiede
`supervised` (0)	positivi con evidenza nota	le frasi di G sopra $s_\star$, le altre sotto
`mil_neg` (1)	negativi	**nessuna** frase sopra $s_\star$
`mil_pos` (2)	positivi senza evidenza (non usato di default)	almeno una frase sopra $s_\star$, ma poche
`none` (3)	WiCE parziali (già tolte dal mix)	niente

La loss del batch (`train_head.batch_loss`) è la media pesata sulle righe che hanno un termine:

$$\mathcal{L}_{batch} = \frac{\sum_b w_b \, \ell_b}{\sum_b w_b \, \mathbb{1}[\ell_b \text{ attivo}]}.$$

Con la loss di riferimento sono attive solo le righe `supervised`; con ZLPR tutte tranne `none`.

5.2 Loss di riferimento (H0): per frase, con soglia fissa (`ref_loss`)

Ogni frase è un sì/no indipendente. Con $y_k = \mathbb{1}[k \in G]$ e $\mathrm{BCE}(s, y) = -y \log \sigma(s) - (1 - y) \log(1 - \sigma(s))$:

$$\ell_{ref} = \frac{1}{n} \sum_k c_k \, \mathrm{BCE}(s_k, y_k) \; + \; \gamma \, \frac{1}{|G|} \sum_{j \in G} \mathrm{softplus}\Big(\mu - s_j + \max_{i \notin G} s_i\Big), \qquad c_k = \begin{cases} \alpha & k \in G \\ \omega & k \notin G \end{cases}$$

con $\alpha = 10$, $\gamma = 0{,}5$ e $\mu = 1$:

- α: le frasi gold sono poche (il 5,4%), e senza peso la testa imparerebbe a dire sempre "no";
- **termine di ordinamento**: ogni frase gold deve superare di un margine μ la migliore non gold.

In uso, la frase è mostrata se $\sigma(s_k) > \tau_e$, con τ_e scelto sul dev (09 §3). Qui $s_\star$ non si usa.

5.3 Loss principale: ZLPR con soglia del documento (`zlpr_loss`)

Con $d_k = s_k - s_\star$ (distanza dalla soglia):

$$\ell_{sup} = \log\Big(1 + \sum_{i \notin G} \omega_i \, e^{d_i}\Big) + \log\Big(1 + \sum_{j \in G} e^{-d_j}\Big),$$

$$\ell_{neg} = \mu_{neg} \, \log\Big(1 + \sum_i e^{d_i}\Big), \qquad \ell_{pos} = \mu_{pos} \Big[\log\big(1 + e^{-\mathrm{LSE}_i \, d_i}\big) + \rho \, \frac{1}{n} \sum_i \sigma(d_i)\Big],$$

con $\mu_{neg} = \mu_{pos} = 0{,}3$, $\rho = 0{,}1$ e $\mathrm{LSE}_i \, x_i = \log \sum_i e^{x_i}$. Nel codice ogni termine è scritto come $\mathrm{softplus}(\mathrm{LSE}(\cdot))$, numericamente stabile: $\log(1 + \sum_i e^{x_i}) = \mathrm{softplus}(\mathrm{LSE}_i \, x_i)$, e ω entra come $+ \log \omega_i$ dentro il LSE.

Figura 8 — La loss ZLPR sull'esempio numerico del test (40 frasi, evidenza = frasi 1 e 40, $\omega = 1$). Prima: la gold 40 è sotto la soglia e una frase simile al claim è sopra, $L = 2{,}526$. Dopo: tutte dal lato giusto, $L = 1{,}473$.

Perché funziona.

1. **È un hinge morbido sul caso peggiore.** Siccome $\log(1 + \sum_i e^{x_i}) \approx \max(0, \max_i x_i)$,

$$\ell_{sup} \approx \big[\max_{i \notin G} d_i\big]_+ + \big[-\min_{j \in G} d_j\big]_+.$$

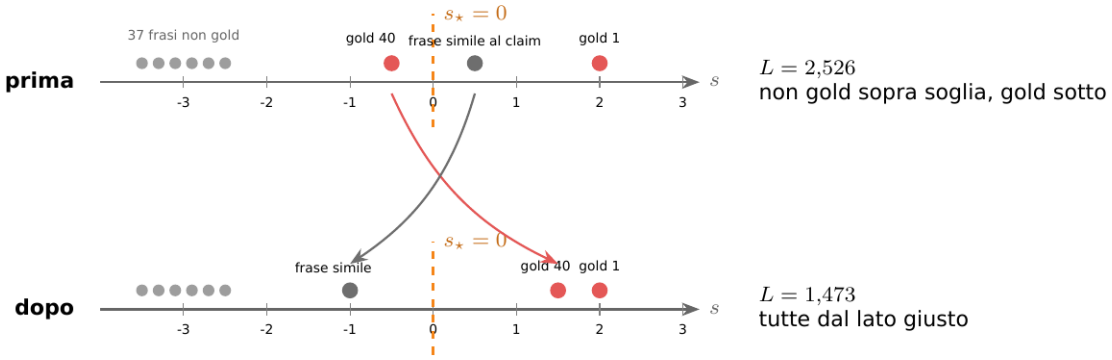


Figura 8: ZLPR

La loss è circa 0 solo quando **tutte** le non gold stanno sotto $s_\star$ e **tutte** le gold sopra. Per l'evidenza sparsa conta proprio questo: una gold sotto soglia (metà evidenza) costa quanto una non gold sopra.

2. **Il gradiente va a chi sbaglia di più.** Per una non gold e una gold:

$$\frac{\partial \ell_{sup}}{\partial s_i} = \frac{\omega_i e^{d_i}}{1 + \sum_{i' \notin G} \omega_{i'} e^{d_{i'}}} \quad (i \notin G), \qquad \frac{\partial \ell_{sup}}{\partial s_j} = -\frac{e^{-d_j}}{1 + \sum_{j' \in G} e^{-d_{j'}}} \quad (j \in G).$$

È una softmax in cui la soglia è una "classe" con logit 0: la frase non gold più sopra la soglia riceve la spinta maggiore verso il basso. Le frasi già ben sotto soglia ricevono un gradiente quasi nullo, quindi le 45 frasi irrilevanti di un documento non diluiscono il segnale (con la BCE invece contano tutte).

3. **La soglia si sposta da sola.** $\partial \ell_{sup}/\partial s_\star = -\sum_{i \notin G} \partial \ell/\partial s_i - \sum_{j \in G} \partial \ell/\partial s_j$: $s_\star$ sale se pesano di più le non gold sopra soglia, scende se pesano di più le gold sotto. Il numero di frasi mostrate si adatta al documento.

4. $\omega < 1$ abbassa la penalità per le frasi non gold sopra soglia. Dove le annotazioni sono incomplete (WiCE, $\omega = 0{,}3$), una frase valida ma non annotata non viene punita troppo.

5. `mil_neg` è lo stesso primo termine con tutte le frasi. Sui claim non supportati nessuna frase deve superare la soglia, anche se somiglia al claim. È ciò che fa scendere i "negativi accesi" (sul campione di 300 righe da 0,95 con H0 a 0,35 con H2c).

Rischio. $x_\star$ conosce il verdetto, quindi con `mil_neg` la testa può imparare "se il verdetto è no, alza la soglia". È coerente con l'obiettivo, perché l'evidenza si mostra solo sui supportati. Sui positivi **incerti** (P appena sopra 0,5), però, la testa può spegnersi. Per questo si misura `recall_uncertain` (10) e, se serve, si abbassa μ_{neg}.

5.4 Termine per le coppie (H-PAIR, `pair_loss`, peso 0,2)

Sulle righe con almeno 2 frasi gold:

$$\ell_{pair} = \underbrace{\mathrm{BCE}(S_{kj}, 1)_{k \neq j \in G}}_{\text{coppie gold}} + \underbrace{\mathrm{BCE}(S_{kj}, 0)_{k \in G,\ j \in \mathrm{top}_5^{S_{k\cdot}}(\bar{G})}}_{\text{per ogni gold, le 5 non gold più "vicine"}}.$$

5.5 Esempi numerici (test automatici in `tests/test_evidence_head.py`)

Caso	Valore
`ref_loss`, 40 frasi, gold = {1, 40}, tutte con $\sigma(s) = 0{,}02$, $\alpha = 10$, $\omega = 1$, solo BCE: $(10 \cdot 2 \cdot 3{,}912 + 38 \cdot 0{,}020)/40$	**1,975**
`zlpr_loss` "prima" (fig. 8): $\log(1 + e^{0,5} + 37e^{-3}) + \log(1 + e^{-2} + e^{0,5}) = 1{,}502 + 1{,}024$	**2,526**
`zlpr_loss` "dopo": $\log(1 + e^{-1} + 37e^{-3}) + \log(1 + e^{-2} + e^{-1,5}) = 1{,}166 + 0{,}306$	**1,473**
`mil_neg` con tutte le frasi a $d = -20$	$< 10^{-6}$
riga `none`	loss 0, gradiente 0

6. Codice

Funzione / classe	File	Ruolo
`EvidenceHead.__init__`, `forward`, `mixed`, `layer_weights`	`head.py`	stadi A–D, H-PAIR
`build_head`, `load_run`	`head.py`	da configurazione / da run
`ref_loss`, `zlpr_loss`, `pair_loss`, `TERMS`	`losses.py`	§5
`select`	`metrics.py`	regola di selezione
test	`tests/test_evidence_head.py`	forme, riempimento, esempi numerici, gradiente zero

Passo 9 — Training della testa (fase 1a)

In breve

`evidence/train_head.py` addestra **solo la testa**, sulle feature salvate (06). Il decision model non è in memoria, quindi il verdetto non può cambiare. Una prova dura minuti. Ogni esperimento è un file in `evidence/configs/` (11).

```
F=data/evidence/features/knowline-4b
python -m evidence.train_head --config evidence/configs/h0.json --train $F/ev_head --overfit 64   # controllo
python -m evidence.train_head --config evidence/configs/h2c.json --train $F/ev_head \
    --dev $F/dev_evid_select --confirm $F/dev_evid_confirm --dev-neg $F/dev_evid_neg [--seed 14 --tag seed14]
```

Le fasi 1b (modello in memoria, media pesata dei token) e 2 (LoRA del modello insieme alla testa) del piano **non sono implementate** (14).

1. La configurazione di un esperimento

```
{"id": "h2c", "description": "H2c: H2b + token mean of the sentence (full head)",
 "layers_read": [19, 23, 27, 31], "parts": ["end", "mean"],
 "head":  {"d": 256, "layers": 2, "heads": 4, "use_mix": true, "use_mean": true, "use_att": true,
           "use_pos": true, "use_pair": false, "att_dropout": 0.1, "dropout": 0.1},
 "loss":  {"name": "zlpr", "mu_neg": 0.3, "mu_pos": 0.3, "rho": 0.1},
 "train": {"epochs": 10, "batch": 64, "lr": 0.001, "weight_decay": 0.01, "patience": 2, "seed": 13,
           "terms": ["supervised", "mil_neg"]}}
```

- `layers_read` e `parts` decidono **che cosa si legge dal disco** (`load_rows`), quindi anche quanta memoria serve: H0 legge un tensore su otto.
- `terms` decide quali righe del mix entrano: H0 e H-LOSS usano solo `supervised`, da H-MIL in poi anche `mil_neg`.

2. Il ciclo (`train`)

1. **Righe in RAM.** Si leggono le righe del mix con i soli layer e parti della configurazione (`load_rows`).
2. **Batch per lunghezza** (`length_batches`). Le righe si ordinano per numero di frasi e si raggruppano a 64, così il riempimento è minimo. L'ordine dei gruppi è casuale e cambia a ogni epoca (seme $13 +$ epoca).
3. **Riempimento** (`collate`): ogni batch si porta alla riga con più frasi; la maschera segna le frasi vere.
4. **Loss** (`batch_loss`): `ref_loss` o `zlpr_loss` per riga, moltiplicata per w_b, più $0{,}2\cdot$ `pair_loss` con H-PAIR (08 §5).
5. **Ottimizzazione**: AdamW (weight decay 0,01), clipping della norma del gradiente a 1. Il learning rate al passo κ su K passi totali, con riscaldamento sui primi $W = \max(1, \lfloor 0{,}05K \rfloor)$ passi, è

$$\eta_\kappa = \eta_0 \cdot \min\Big(1, \frac{\kappa+1}{W}\Big) \cdot \frac{1}{2}\Big(1 + \cos\frac{\pi \min(\kappa, K)}{K}\Big), \qquad \eta_0 = 10^{-3}.$$

6. **A fine epoca**: i punteggi sul dev (`scores_of`), poi la scelta della regola sul dev (`choose_rule`, §3), poi le metriche. Il criterio del checkpoint è

$$\text{score} = \tfrac{1}{2}\big(\text{F1}_{frase} + \text{insieme completo}\big) \quad \text{su } \texttt{dev_evid_select}.$$

   Si tiene lo stato migliore e ci si ferma dopo 2 epoche senza miglioramento (`patience`).

Iperparametro	Valore
learning rate	10^{-3}, riscaldamento 5%, coseno
weight decay	0,01
batch	64 righe, raggruppate per lunghezza
epoche	fino a 10, arresto dopo 2 senza miglioramento
dropout	0,1 nel transformer; 0,1 per frase sulle feature di attenzione
seme	13 (per la configurazione finale anche 14 e 15, con `--seed` e `--tag`)

3. La regola di selezione (`choose_rule`)

Si fissa **dopo** il training, su `dev_evid_select`. Si valutano solo i positivi che il verdetto giudica supportati (10 §1).

Loss	Regola	Griglia	Scelta
`ref`	$\sigma(s_k) > \tau_e$, cioè soglia $\text{logit}(\tau_e)$	$\tau_e \in \{0{,}05, \dots, 0{,}95\}$	F1 migliore
`zlpr`	$s_k > s_\star + \delta$	$\delta \in \{-1{,}0; -0{,}9; \dots; 1{,}0\}$	F1 migliore, ma $\delta \neq 0$ solo se guadagna almeno 1 punto

Sempre almeno 1 frase e al massimo 5. La regola va in `selection.json`, la confermano le metriche su `dev_evid_confirm`, e la usano valutazione e server.

4. Che cosa scrive una run

`data/evidence/runs/knowline-4b/<id>[_<tag>]/:`

File	Contenuto
`config.json`	la configurazione, le cartelle di feature, il loro `meta.json`, D
`head.safetensors`	i pesi dell'epoca migliore
`selection.json`	la regola scelta
`log.jsonl`	per epoca: loss, F1, insieme completo, score, regola, secondi, pesi π dei layer
`dev_metrics.json`	metriche complete su select, confirm e neg (10)

E una riga in `data/evidence/runs/registry.md` (11).

Il training rifiuta dev e train estratti con impostazioni diverse (`same_features`: modello, revisione, preambolo, layer, domanda, hash del codice).

5. Che cosa guardare

Cosa	Dove	Problema	Cosa fare
F1 e insieme completo	`log.jsonl`	non superano E-LEX dopo 2 epoche	controllare il rendering (06 §5) prima della testa
F1 per distanza in frasi	`dev_metrics.json`: `f1_by_sent_distance`	crollo oltre 10 frasi	H3c (4 layer), H-PAIR
F1 per distanza in token	`f1_by_tok_distance`	crollo oltre 1.024 o 4.096	H2b (attenzione)
frasi mostrate	`shown_mean`, `count_error_mean`	molte più o molte meno delle gold	δ, ω
negativi accesi	`dev_evid_neg` → `negatives_lit`	oltre 0,30	μ_{neg} più alto (`h_mil1`)
recall sui positivi incerti	`recall_uncertain`	molto sotto la media	μ_{neg} più basso
pesi π dei layer	`log.jsonl` → `layer_weights`	un layer sotto il 10%	toglierlo (meno disco, meno costo)

6. Controlli prima di ogni nuova configurazione

Controllo	Atteso	Misurato (2026-11-04)
`--overfit 64`: la testa impara a memoria 64 righe (30 epoche, batch 16)	score ≈ 1	1,000 con H0 e con H2c
righe `none`	loss e gradiente 0	sì (test)
riempimento	stessi punteggi da sola o in batch	differenza < 10^{-4} (test)

Prova end-to-end su 300 righe del mix (circa 190 positivi) e 300 di `dev_evid_select`: F1 0,66 (H0), 0,68 (H-MIL), 0,78 (H2c); negativi accesi 0,95, 0,40, 0,35. Sono numeri da campione minuscolo: indicano che la pipeline impara, non quanto è buona la testa.

7. Codice

Funzione	Ruolo
`collate`, `length_batches`	batch riempiti e raggruppati per lunghezza
`run_head`, `batch_loss`	forward e loss del batch

Funzione	Ruolo
`scores_of`, `predictions`	punteggi → predizioni nel formato delle metriche
`choose_rule`	τ_e o δ sul dev
`load_rows`, `train`	il ciclo con arresto anticipato
`append_registry`, `main`	registro e salvataggio della run

Passo 10 — Valutazione e latenza

In breve

Per ogni testa si misurano tre cose: 1. le **frasi mostrate** sono quelle giuste, anche lontane fra loro o dall'inizio del documento; 2. il **verdetto** non è cambiato; 3. la **latenza** resta quella di un decision model.

Codice: `evidence/metrics.py` (le metriche, comuni a baseline, training e valutazione), `evidence/eval_evidence.py` e `evidence/latency.py`.

```
python -m evidence.eval_evidence --run data/evidence/runs/knowline-4b/<id> --features $F/dev_evid_select \
    --rows dataset/evidence/splits/dev_evid_select.jsonl --sufficiency 300
python -m evidence.latency --run data/evidence/runs/knowline-4b/<id> --n 1000
```

1. Su quali righe

Il sistema mostra frasi solo se $P(\text{supportato}) > 0{,}5$. L'evidenza si misura quindi sulle righe

$$\mathcal{E} = \{\, b : y_b = 1,\ G_b \neq \emptyset,\ P_b > 0{,}5 \,\}.$$

Le righe positive con $P_b \leq 0{,}5$ si contano a parte (`missed_by_verdict`).

Tutte le metriche partono dallo stesso formato di **predizione**, uno per riga: `id`, `source`, `label`, `p_supported`, `scores`, `threshold`, `shown`, `evidence_sets`, `gold_tok_last`. Le producono `train_head.predictions` per la testa e `baselines.make_preds` per le baseline.

2. Metriche dell'evidenza (`summarize`)

Per una riga con frasi mostrate $\mathcal{S}$, insiemi gold alternativi $G^{(1)}, \dots, G^{(m)}$ e unione $G = \bigcup_r G^{(r)}$ (`sentence_metrics`):

$$\mathrm{P} = \frac{|\mathcal{S} \cap G|}{|\mathcal{S}|}, \quad \mathrm{R} = \frac{|\mathcal{S} \cap G|}{|G|}, \quad \mathrm{F1} = \frac{2\,\mathrm{P}\,\mathrm{R}}{\mathrm{P} + \mathrm{R}},$$

$$\text{completo} = \mathbb{1}\big[\exists r :\ G^{(r)} \subseteq \mathcal{S}\big], \quad \text{esatto} = \mathbb{1}\big[\exists r :\ G^{(r)} = \mathcal{S}\big].$$

Esempio: con $\mathcal{S} = \{1, 4\}$ e insiemi gold $\{1, 4\}$ e $\{7\}$ si ha P = 1, R = 2/3 e F1 = 0,8; l'insieme è completo ed esatto.

Metrica (chiave)	Definizione	Perché
`precision`, `recall`, `f1`	media su $\mathcal{E}$	qualità per frase
`full_set`	media di "completo"	**per l'evidenza multi-frase è quella che conta**: metà evidenza non convince
`exact`	media di "esatto"	
`auprc`	area sotto la curva precisione–recall su tutte le frasi di $\mathcal{E}$	qualità dell'ordinamento, indipendente dalla soglia
`count_error_mean`, `shown_mean`	$\lvert\mathcal{S}\rvert - \lvert G\rvert$; $\lvert\mathcal{S}\rvert$	troppe o troppo poche frasi

Metrica (chiave)	Definizione	Perché
`f1_by_sent_distance`	F1 per bucket di $\max G - \min G$: 0–1 / 2–9 / 10–19 / ≥ 20	evidenza sparsa
`f1_by_tok_distance`	F1 per bucket di `gold_tok_last` $= b_{\max G}$ − `doc_tok_start`: < 1.024 / 1.024–4.096 / > 4.096	evidenza in fondo a contesti lunghi
`f1_far_10plus`, `f1_beyond_1024_tok`	F1 con gold a ≥ 10 frasi; con l'ultima gold oltre 1.024 token	i numeri del registro
`recall_uncertain`	recall sulle righe con $0{,}5 < P \le 0{,}65$	effetto di `mil_neg` sui positivi incerti
`negatives_lit`	quota di negativi con almeno una frase sopra la propria soglia	la testa non deve accendere frasi solo perché somigliano al claim
`score`	$\frac{1}{2}(\text{F1} + \text{completo})$	criterio del checkpoint
`by_source`	F1 e completo per fonte	

2.1 Sufficienza e necessità (`eval_evidence.sufficiency`, GPU)

Non dipendono dalle annotazioni, che possono essere incomplete. Sia $p(\cdot)$ il verdetto del modello su un documento ridotto (`extract.rebuild`):

$$\text{sufficienza} = \frac{1}{|\mathcal{E}|}\sum_{b\in\mathcal{E}} \mathbb{1}\big[p(\mathcal{S}_b) > 0{,}5\big], \qquad \text{necessità} = \frac{1}{|\mathcal{E}|}\sum_{b\in\mathcal{E}} \mathbb{1}\big[p(\overline{\mathcal{S}_b}) \le 0{,}5\big].$$

- **Sufficienza**: le sole frasi mostrate bastano a far dire "supportato".
- **Necessità**: senza di esse il modello non dice più "supportato". Si riporta soltanto, perché fallisce anche con evidenza corretta quando l'informazione è ripetuta nel documento.

Servono 2 forward per riga, e si calcola solo in valutazione. Sul campione di 20 righe: sufficienza 0,90, necessità 0,65.

3. Metriche del verdetto (`verdict_metrics`)

Su tutte le righe del set, con $\hat{y}_b = \mathbb{1}[P_b > 0{,}5]$:

$$\text{BAcc} = \frac{1}{2}\Big(\frac{\text{TP}}{\text{TP}+\text{FN}} + \frac{\text{TN}}{\text{TN}+\text{FP}}\Big), \qquad \text{ECE} = \sum_{c=1}^{10} \frac{|B_c|}{N}\Big|\underbrace{\tfrac{1}{|B_c|}\textstyle\sum_{b\in B_c} y_b}_{\text{accuratezza}} - \underbrace{\tfrac{1}{|B_c|}\textstyle\sum_{b\in B_c} P_b}_{\text{confidenza}}\Big|,$$

dove B_c sono le righe con P nel decimo c. L'AUC si calcola con `roc_auc_score`.

Nella fase 1a il modello è congelato, quindi il verdetto **deve** coincidere con quello senza testa: se non coincide, c'è un errore nella pipeline. Le stesse metriche servono a `check_preamble` per confrontare i preamboli (05 §4).

4. Che cosa si salva

`<run>/eval/<set>/metrics.json` (`evidence`, `verdict`, `rule`, `sufficiency`) e `predictions.jsonl`: per ogni riga, punteggi, soglia, frasi mostrate e frasi gold. Le metriche si possono ricalcolare senza rifare i forward.

I test (`test_evid_id`, `test_evid_ood`) si misurano **una volta sola**, a configurazione congelata, sulla release (12). Le loro feature si estraggono a parte.

5. Latenza (`latency.py`)

Stesso processo e stessa GPU, con richieste reali (di default dalle righe di `test_evid_id`; le prime 50 non si contano). Tre condizioni:

	Prompt	Uscita
A	senza preambolo	solo verdetto
B	con preambolo	solo verdetto
C	con preambolo	verdetto + evidenza (hook, righe di attenzione, testa, selezione): ciò che fa il server

Obiettivo: $C - B \leq \max(0{,}03\,B,\ 2\,\text{ms})$. Si misurano a parte anche la divisione in frasi e la testa.

Misurato su RTX 5090 Laptop, 40 richieste di `dev_evid_select` (prova):

		p50	p95
A		322,9 ms	480,4 ms
B		337,0 ms	484,7 ms
C		338,9 ms	490,3 ms
divisione in frasi		1,3 ms	1,8 ms
testa + selezione		1,1 ms	1,2 ms
C − B	**+1,8 ms (+0,5%)**, entro l'obiettivo		
B − A (costo del preambolo)		+14,1 ms	

6. Codice

Funzione	File	Ruolo
`select`, `sentence_metrics`, `auprc`	`metrics.py`	selezione e metriche per riga
`sent_bucket`, `tok_bucket`, `gold_tok_last`	`metrics.py`	bucket di distanza
`summarize`	`metrics.py`	tutte le metriche di un set
`verdict_metrics`, `sufficiency`, `main`	`eval_evidence.py`	verdetto, sufficienza, salvataggio
`percentiles`, `main`	`latency.py`	condizioni A/B/C

Non implementato: le prove di comportamento della doc del piano 10 §4 (evidenza spostata, frasi di disturbo, frasi riscritte), che dipendono dal passo 03 (03).

Passo 11 — Esperimenti

In breve

Si parte da una testa semplice (**H0**) e si aggiunge **un pezzo alla volta**, tenendolo solo se migliora. Tutto si fa su KnowLine-4B, con le feature salvate (fase 1a): una prova è un training della sola testa, di pochi minuti.

Un esperimento sulla **testa** o sulla **loss** è un file in `evidence/configs/`. Un esperimento sui **dati** (pesi, ω, negativi) è una ricetta in `evidence/recipes/` più un nuovo mix, da estrarre di nuovo.

1. Regole di decisione

1. Una modifica per prova.
2. Si decide su `dev_evid_select` con quattro numeri: F1 per frase, insieme completo, F1 con evidenza a ≥ 10 frasi, F1 con evidenza oltre 1.024 token. In più si guardano i negativi accesi su `dev_evid_neg`.

3. **Una modifica si tiene** se aumenta l'F1 di almeno 1 punto o l'insieme completo di almeno 2, senza far scendere gli altri numeri di più di 1 punto.
4. A parità di risultato, si tiene la versione più semplice.
5. Guadagni sotto i 2 punti si ripetono con altri 2 semi: `--seed 14 --tag seed14`, `--seed 15 --tag seed15`.

Il registro `data/evidence/runs/registry.md` è scritto in automatico da `train_head.append_registry`, con una riga per run: run, config, feature, F1, insieme completo, F1 ≥ 10 frasi, F1 > 1.024 token, negativi accesi, descrizione. La colonna **decision** la compila chi decide.

2. Le configurazioni pronte

File	Che cosa cambia rispetto al precedente	Layer	Parti	Loss	Righe	Che cosa si guarda
`h0.json`	testa di partenza: niente miscela, niente media, niente attenzione	[31]	end	`ref`	supervised	batte E-LEX ed E-ATT? Se no, cercare errori nel rendering
`h_loss.json`	soglia del documento (ZLPR)	[31]	end	`zlpr`	supervised	insieme completo, frasi mostrate
`h_mil.json`	+ negativi, $\mu_{neg} = 0{,}3$	[31]	end	`zlpr`	+ mil_neg	negativi accesi, `recall_uncertain`
`h_mil1.json`	idem, $\mu_{neg} = 1$	[31]	end	`zlpr`	+ mil_neg	idem
`h2a.json`	+ 4 layer miscelati	19–31	end	`zlpr`	+ mil_neg	F1; pesi π
`h2b.json`	+ attenzione att_k	19–31	end	`zlpr`	+ mil_neg	F1 lontano; frasi "ponte"
`h2c.json`	+ media dei token (**testa completa**)	19–31	end, mean	`zlpr`	+ mil_neg	F1 sulle frasi lunghe
`h3a.json`	testa completa **senza** transformer	19–31	end, mean	`zlpr`	+ mil_neg	atteso un crollo su HoVer e sull'evidenza lontana
`h3c1.json`, `h3c4.json`	1 o 4 layer di transformer	19–31	end, mean	`zlpr`	+ mil_neg	quanta capacità serve
`h9.json`	senza vettore di posizione	19–31	end, mean	`zlpr`	+ mil_neg	preferenza per inizio o fine
`h_pair.json`	+ punteggio per coppie, peso 0,2	19–31	end, mean	`zlpr` + pair	+ mil_neg	solo se l'insieme completo su HoVer resta < 0,65

Le configurazioni H3a, H3c, H9 e H-PAIR partono dalla testa completa. Se gli esperimenti principali scelgono una testa diversa, vanno adattate.

3. Esperimenti che richiedono altre feature o un altro mix

Nome	Come si fa
H4 (senza preambolo)	`python -m evidence.extract --rows <mix> <dev...> --preamble none --suffix _nopre`, poi `train_head` con `--train $F/ev_head_nopre --dev $F/dev_evid_select_nopre`
H7 (ω = 0,3 / 0,5 / 1)	copiare `recipes/ev_head.json`, cambiare `omega`, fare un nuovo `make_mix`. Le feature **non** dipendono da ω, che sta solo nell'indice: basta ri-estrarre il nuovo mix oppure modificare `omega` in `index.jsonl`
H12 (peso di `db_split` = 0 / 0,5 / 1)	come H7, con `row_weight` della famiglia `hover_db_split`
quota di negativi	`neg_fraction` nella ricetta

Siccome `row_weight` e `omega` stanno in `index.jsonl` e non negli shard, un esperimento sui pesi si può fare senza ri-estrarre: basta riscrivere l'indice con i nuovi pesi.

4. Sequenza consigliata

```
E-RAND, E-LEX, E-ATT (baselines.py) → H0 → H-LOSS → H-MIL (0,3 / 1) → H2a → H2b → H2c
   → controlli sulla migliore: H3a, H3c1/H3c4, H9, H4, H7, H12 → (H-PAIR se serve)
   → 3 semi della finale → E-ABL su 100 righe per fonte (confronto C3) → rilascio → test una volta
```

5. Che cosa NON si può fare con questa implementazione

- H2d (media pesata dei token decisa dalla testa) richiede la fase 1b, con il modello in memoria;
- H6 ed E6 (fase 2, LoRA del modello) non sono implementati;
- H5 e H8 (dati generati e spostati) dipendono dal passo 03;
- E0 e X0 (elephant-1 e gli altri modelli) sono fuori perimetro (14).

Passo 12 — Rilascio e server

In breve

- `evidence/release.py` crea una cartella `releases/<nome>/` con tutto ciò che serve al server, congelato con degli hash.
- `evidence/serve.py` fa, per ogni richiesta, **un forward** e restituisce verdetto ed evidenza. Usa lo stesso codice della valutazione: ciò che si è misurato è ciò che gira.

```
python -m evidence.release --run data/evidence/runs/knowline-4b/<id> --name knowline-4b-fc-ev-v1
python -m evidence.release --verify releases/knowline-4b-fc-ev-v1
python -m evidence.serve --run releases/knowline-4b-fc-ev-v1 --port 8090
curl -s localhost:8090/v1/evidence -d '{"document": "...", "claim": "..."}'
python -m evidence.serve --run releases/knowline-4b-fc-ev-v1 --demo
```

1. Una richiesta (`Service.answer`)

$$\text{documento} \xrightarrow{\text{split}} \{[s_k, e_k)\} \xrightarrow{\text{render}} (\texttt{input_ids},\ [a_k, b_k])$$
$$\xrightarrow{\text{forward_sentences}} (\ell_A, \ell_B,\ q,\ h^{end},\ h^{mean},\ att)$$
$$\xrightarrow{P>0{,}5\ ?} \text{testa} \to (s, s_\star) \xrightarrow{\text{select}} \mathcal{S}$$

1. Divisione in frasi con `evidence/sentences.py`, lo stesso divisore del training.
2. Prompt con il preambolo con cui la testa è stata addestrata (`feature_meta.preamble` della run) e posizioni delle frasi.
3. Un forward con gli hook (`to_cpu=False`: le feature restano sulla GPU).
4. Se $P \leq 0{,}5$: risposta senza evidenza.
5. Altrimenti: la testa sui layer della configurazione (`layers_read`), poi la regola della run (δ o τ_e), poi le posizioni nel **documento originale**.

```
{"p_supported": 0.9987, "supported": true,
 "evidence": [{"sentence": 0, "start": 0, "end": 77, "score": 0.8333, "text": "Saint Paul School of Theology is
              {"sentence": 4, "start": 185, "end": 259, "score": 0.6102, "text": "Nancy R. Howell is ..."}]}
```

Regole del servizio: - `start` / `end` si riferiscono al documento così come l'ha mandato il client; - `score` vale $\sigma(s_k - \theta)$; - un documento oltre 32.768 token (`config.MAX_TOKENS_SERVE`) dà l'errore `document_too_long` e non viene mai troncato; un documento vuoto dà `empty_document`; - un lock serializza i forward sulla GPU, con batch 1; - endpoint HTTP: `POST /v1/evidence`, `GET /health` (`make_handler`, `ThreadingHTTPServer`).

2. La cartella di rilascio (`release.build`)

File	Contenuto
`head.safetensors`, `config.json`, `selection.json`	la testa, la sua configurazione (impostazioni delle feature comprese), la regola di selezione
`decision.json`	tutte le impostazioni: modello e revisione, domanda, preambolo, soglia, layer, feature, selezione, divisore, limite di token
`sentences/`	copia di `sentences.py` e dei parametri inglesi di NLTK `punkt_tab`
`frozen.json`	sha256 della testa, di `sentences.py`, `render.py`, `model.py`, `config.py`, di ogni file di `punkt_tab`, e dei manifest di split e mix; commit git
`report.md`	metriche della run (dev, valutazioni, latenza, se ci sono)
`MODEL_CARD.md`	che cosa fa e che cosa non fa

I pesi di KnowLine non si copiano: sono quelli della revisione indicata in `decision.json`. Una release non si sovrascrive mai.

3. Perché il divisore va rilasciato con il modello

La testa lavora su **frasi**. Se in produzione il documento fosse diviso anche di poco diversamente, la testa vedrebbe unità diverse da quelle su cui è stata addestrata, e gli indici restituiti non corrisponderebbero più. Il risultato di `split` dipende dal nostro codice, dalla versione di NLTK e dai file `punkt_tab`. All'avvio da una release, `Service` chiama `release.verify`, che ricalcola gli hash sulla macchina e si ferma se qualcosa è diverso. Lo stesso vale per `render.py` e `model.py`: cambiarli cambierebbe le posizioni o le feature.

4. Controlli prima del rilascio

Controllo	Deve dare	Misurato (prova, 2026-11-04)
Server = valutazione, stesse righe	stesse frasi; $\lvert\Delta P\rvert < 0{,}005$	60/60 righe, $\Delta P = 0$
`release --verify`	hash uguali	sì
Latenza	C − B ≤ 3% o 2 ms	+1,8 ms (+0,5%)
Test (`test_evid_id`, `test_evid_ood`)	una volta sola, sulla release	da fare

5. Codice

Funzione / classe	File	Ruolo
`Service.__init__`, `Service.answer`	`serve.py`	caricamento (con verifica) e una richiesta
`make_handler`, `main`	`serve.py`	HTTP e `--demo`
`build`, `hashes`, `verify`, `punkt_dir`, `model_card`	`release.py`	cartella di rilascio e hash

Comandi in ordine, checklist e problemi frequenti

1. Comandi in ordine

```
cd <repo>
F=data/evidence/features/knowline-4b; S=dataset/evidence/splits

# 0. test (CPU, secondi)
for t in tests/test_evidence_*.py; do python $t; done

# 1. split e mix (CPU, ~10 s) — già fatti il 2026-11-04
python -m evidence.data.make_splits
python -m evidence.data.make_mix --recipe evidence/recipes/ev_head.json

# 2-3. controlli su rendering e modello
python -m evidence.render 400                  # CPU: frasi = token
python -m evidence.model --selftest            # GPU: verdetto, riassunti, attenzione

# 4. preambolo (GPU, ~1.200 righe x 3 forward, ~25 min) — PRIMA dell'estrazione
python -m evidence.check_preamble

# 5. estrazione (GPU, ~2,5 righe/s: mix ~2,3 h, dev ~8 min; ~45 GB)
python -m evidence.extract --rows dataset/evidence/mixes/ev_head.jsonl $S/dev_evid_select.jsonl \
    $S/dev_evid_confirm.jsonl $S/dev_evid_neg.jsonl

# 6. baseline
python -m evidence.baselines --features $F/dev_evid_select --rows $S/dev_evid_select.jsonl --method rand lex att
python -m evidence.baselines --features $F/dev_evid_select --rows $S/dev_evid_select.jsonl --method abl --per-so

# 8. training: controllo, poi esperimenti uno alla volta
python -m evidence.train_head --config evidence/configs/h0.json --train $F/ev_head --overfit 64
for c in h0 h_loss h_mil h2a h2b h2c; do
  python -m evidence.train_head --config evidence/configs/$c.json --train $F/ev_head \
      --dev $F/dev_evid_select --confirm $F/dev_evid_confirm --dev-neg $F/dev_evid_neg
done
cat data/evidence/runs/registry.md

# 9. valutazione della configurazione scelta (dev)
python -m evidence.eval_evidence --run data/evidence/runs/knowline-4b/<id> --features $F/dev_evid_select \
    --rows $S/dev_evid_select.jsonl --sufficiency 300
python -m evidence.latency --run data/evidence/runs/knowline-4b/<id> --n 1000

# 11. rilascio, poi test una volta sola
python -m evidence.release --run data/evidence/runs/knowline-4b/<id> --name knowline-4b-fc-ev-v1
python -m evidence.extract --rows $S/test_evid_id.jsonl $S/test_evid_ood.jsonl
for t in test_evid_id test_evid_ood; do
  python -m evidence.eval_evidence --run releases/knowline-4b-fc-ev-v1 --features $F/$t --rows $S/$t.jsonl --suf
done
python -m evidence.serve --run releases/knowline-4b-fc-ev-v1 --port 8090
```

2. Checklist

Passo 1–2, dati - [x] dataset costruiti con `--hover-db-fallback`; allineamento ≥ 90% per fonte - [x] split per `doc_id`, nessun documento condiviso; WiCE con `aggrefact_overlap` mai in train - [x] mix `ev_head` con i vincoli rispettati

Passo 2–3, rendering e modello - [x] `python -m evidence.render 400`: tutte le righe corrette - [x] `python -m evidence.model --selftest` superato - [x] `tests/test_evidence_render.py` superato

Passo 4, preambolo - [ ] `check_preamble` lanciato; `preamble_choice.json` scritto; `acceptable` verificato

Passo 5, estrazione - [ ] mix e dev estratti con il preambolo scelto; `report` stampa la quota di evidenza oltre 1.024 token (≥ 0,15) - [ ] `skipped.json` controllato (righe saltate e motivo)

Passo 6–9, baseline, testa, training, valutazione - [ ] baseline su `dev_evid_select` - [x] `tests/test_evidence_head.py` superato - [ ] `--overfit 64` sulle feature vere: score ≈ 1 - [ ] H0 → H2c nel registro, con le decisioni compilate - [ ] configurazione finale con 3 semi; sufficienza ≥ 85%; latenza nell'obiettivo

Passo 11, rilascio - [ ] release creata e verificata; server = valutazione su 200 righe - [ ] test misurati una volta sola; `report.md` aggiornato

3. Se succede…

Problema	Prima cosa da controllare	Poi
La testa non impara (F1 vicino a E-RAND)	`python -m evidence.render` sulle righe del mix	`meta.json` delle feature (preambolo, layer)
`--overfit 64` non arriva a ≈ 1	posizioni delle frasi, etichette `gold` in `index.jsonl`	il codice della loss (`tests/test_evidence_head.py`)
Fa come E-LEX (sbaglia frasi "ponte")	quante righe HoVer ci sono nel mix	H3c4; H-PAIR; `h_mil1`
Bene su SciFact, male su HoVer o con frasi lontane	H3a: il transformer serve?	H3c4, H-PAIR
Mostra troppe frasi	δ in `selection.json`, `count_error_mean`	ω più alto (H7)
Non mostra quasi nulla sui positivi incerti	`recall_uncertain`	μ_{neg} più basso
Mostra spesso la prima frase o il titolo	posizione delle gold nel mix	H9
Precisione bassa ma sufficienza alta	leggere le frasi "sbagliate": spesso sono evidenza valida non annotata	ω più basso per quella fonte
Risultati molto diversi fra semi	dipendenza da poche teste di attenzione	`att_dropout` più alto, o `use_att=False`
"extracted with different settings"	`meta.json` delle due cartelle	ri-estrarre con lo stesso preambolo e lo stesso codice
Il verdetto in `eval_evidence` cambia	non deve succedere nella fase 1a	prompt identico (`tests/test_evidence_render.py`)
Il server rifiuta la release (hash)	`sentences.py`, `render.py`, `model.py`, `punkt_tab` modificati?	ripristinarli, o ri-estrarre e riaddestrare
Out of memory in estrazione	righe oltre 16.000 token (`skipped.json`)	`--max-tokens` più basso
Troppo lento	tempi separati in `latency.json`	meno layer, `use_mean=False`, 1 layer di transformer

Che cosa non è implementato, e da dove partire

L'implementazione copre la pipeline **obbligatoria** del piano per **KnowLine-4B** (decisione del 2026-11-04). Per ogni parte mancante: che cosa servirebbe e quale codice esistente si può riusare.

Parte del piano	Stato	Che cosa servirebbe	Riuso
Altri decision model (elephant-1, Jeffan, d3-flash, OneJev, Nimble, StartLux, pplx, Clef)	no	per ogni modello: un renderer con il suo prompt (stato in JSON → la mappa `escape_with_map` della doc del piano 06 §3) e, per i bidirezionali, la loro maschera dentro `sdpa_capture` (`CAPTURE["inner"]`)	`render.token_char_spans` e `sentence_token_spans` sono già generici; `model.SentenceTap` e `attention_per_sentence` funzionano su qualunque Qwen3.5
Dati generati (passo 03)	no	generatore LLM + giudice; ablazione; evidenza spostata	`baselines.ablation_group`, `extract.rebuild`, ricetta del mix (03)
`test_evid_sparse` e prove di comportamento	no	evidenza spostata nei test	`extract.rebuild`
Fase 1b (H2d: media pesata dei token)	no	training con il modello in memoria e i vettori di tutti i token della frase	`SentenceTap` andrebbe esteso per restituire i token
Fase 2 (LoRA del modello + testa)	no	gradiente attraverso il modello; controllo di BAcc ed ECE a ogni checkpoint	`forward_sentences` senza `no_grad`
E-EMB	no	`sentence-transformers` + `BAAI/bge-small-en-v1.5`	`baselines.evaluate_method`
Sonda per layer (doc del piano 15 §3)	no	estrazione su tutti i 32 layer per circa 700 righe, poi una regressione logistica per layer	aggiungere layer a `config.SENTENCE_LAYERS` e usare `extract.py`
Importanza controfattuale (doc del piano 14)	no	un forward per frase tolta	`extract.rebuild`, `forward_sentences(features=False)`
Verdetto fine-tunato (LoRA del fine-tuning binario)	non esiste nella repo	caricare la LoRA in `model.load_model`; eventualmente cambiare `config.QUESTION`	il resto della pipeline non cambia

La strada più promettente

La **sonda per layer** costa poco e risponde a una domanda aperta: i layer 19, 23, 27 e 31 sono davvero i migliori? Si ottiene già, in parte, dai pesi π della miscela dopo H2a (08 §2.A).

Il secondo passo naturale sono gli **altri modelli**, a partire da elephant-1, che è bidirezionale nei layer pieni. Lì l'esperimento H4 (preambolo sì o no) diventa interessante, perché le frasi vedono il claim anche senza preambolo.

Riepilogo — tutte le formule in una pagina

Notazione in 00 §3. Fra parentesi il file e la funzione che realizzano ogni formula.

1. Dal documento alle feature (un forward)

Frasi (`sentences.split`): documento → $\{[s_k, e_k)\}_{k<n}$.

Token delle frasi (`render.sentence_token_spans`): con γ_t l'ultimo carattere non spazio del token t, riportato al testo originale,

$$\phi(t) = k \iff s_k \le \gamma_t < e_k, \qquad a_k = \min \phi^{-1}(k), \quad b_k = \max \phi^{-1}(k), \quad n_k = b_k - a_k + 1.$$

Verdetto (`model.p_supported`):

$$P(\text{supportato}) = \sigma(\ell_A - \ell_B), \qquad \ell = \log \text{softmax}\big(W_U \,\text{RMSNorm}(H_T^{(31)})\big).$$

Riassunti delle frasi (`model.SentenceTap`), per $l \in \{19, 23, 27, 31\}$:

$$h_{k,l}^{end} = H_{b_k}^{(l)}, \qquad h_{k,l}^{mean} = \frac{1}{n_k} \sum_{i=a_k}^{b_k} H_i^{(l)}, \qquad q_l = H_T^{(l)}.$$

Attenzione per frase (`model.sdpa_capture`, `attention_per_sentence`), per $l \in \{3, 7, \dots, 31\}$ e $j \in \{0..15\}$:

$$A_i^{(l,j)} = \text{softmax}_i\Big(\frac{\mathbf{q}_T^{(l,j)} \cdot \mathbf{k}_i^{(l,\lfloor j/4 \rfloor)}}{\sqrt{256}}\Big), \qquad att_k[(l,j)] = \sum_{i=a_k}^{b_k} A_i^{(l,j)} \quad \Rightarrow \quad att_k \in \mathbb{R}^{128}.$$

2. La testa (`head.EvidenceHead`)

A. $\pi = \text{softmax}(\alpha), \qquad m(V) = \text{LN}(\sum_l \pi_l V_l)$

B. $e_k = W_h[m(h_k^{end}); m(h_k^{mean})] + W_a[\xi_k \, att_k; \log n_k] + P_k + t_{frase}$
$x_\star = W_q \, m(q) + t_{risposta}$

C. $[X_\star; X_0; \dots; X_{n-1}] = \text{Encoder}^{(2)}_{\text{non causale}}([x_\star; e_0; \dots; e_{n-1}])$

D. $s_k = w_o^\top X_k + b_o, \qquad s_\star = w_o^\top X_\star + b_o$

Il **layer dell'encoder** (pre-norm, 4 teste da 64, FFN 1.024):

$$U = Z + \text{MHA}(\text{LN}_1 Z), \qquad Z' = U + W_2 \,\text{ReLU}(W_1 \,\text{LN}_2 U + b_1) + b_2.$$

Selezione (`metrics.select`), con $\theta = s_\star + \delta$:

$$\mathcal{S} = \text{top}_5\{k : s_k > \theta\} \text{ oppure } \{\arg\max_k s_k\}, \quad \text{solo se } P(\text{supportato}) > 0{,}5.$$

H-PAIR: $S_{kj} = (U_1 X_k)^\top (U_2 X_j)$, e poi $s_k \mathrel{+}= \tau \max_{j \ne k} (S_{kj} + \text{sg}(s_j))$.

Parametri: 3.716.357 per la testa completa, 3.060.994 per H0. Tempo: circa 1,1 ms per documento.

3. Le loss (`losses.py`)

Con $d_k = s_k - s_\star$, G = frasi gold e ω il peso delle non gold:

$$\ell_{sup} = \log\Big(1 + \sum_{i \notin G} \omega_i e^{d_i}\Big) + \log\Big(1 + \sum_{j \in G} e^{-d_j}\Big) \;\approx\; \big[\max_{i \notin G} d_i\big]_+ + \big[-\min_{j \in G} d_j\big]_+$$

$$\ell_{neg} = 0{,}3\,\log\Big(1 + \sum_i e^{d_i}\Big)$$

$$\ell_{pos} = 0{,}3\,\Big[\log\big(1 + e^{-\mathrm{LSE}_i d_i}\big) + 0{,}1 \cdot \tfrac{1}{n}\textstyle\sum_i \sigma(d_i)\Big]$$

$$\ell_{ref} = \frac{1}{n}\sum_k c_k\,\mathrm{BCE}(s_k, y_k) + 0{,}5 \cdot \frac{1}{|G|}\sum_{j \in G} \mathrm{softplus}(1 - s_j + \max_{i \notin G} s_i), \quad c_k \in \{10, \omega\}$$

$$\mathcal{L}_{batch} = \frac{\sum_b w_b\,\ell_b}{\sum_b w_b\,\mathbb{1}[\ell_b \text{ attivo}]}$$

Gradiente di ZLPR: $\partial \ell_{sup}/\partial s_i = \omega_i e^{d_i}/(1 + \sum_{i'} \omega_{i'} e^{d_{i'}})$ per $i \notin G$. È una softmax con la soglia come classe a logit 0: la non gold peggiore riceve la spinta maggiore, quelle già sotto soglia quasi niente.

4. Training (`train_head.py`)

$$\eta_\kappa = 10^{-3} \cdot \min\Big(1, \frac{\kappa + 1}{W}\Big) \cdot \tfrac{1}{2}\Big(1 + \cos\frac{\pi \min(\kappa, K)}{K}\Big), \qquad W = \lfloor 0{,}05K \rfloor$$

checkpoint: max $\frac{1}{2}(\mathrm{F1} + \mathrm{completo})$ su `dev_evid_select`

Regola: $\delta \in [-1, 1]$ a passi di 0,1 (ZLPR), oppure $\tau_e \in [0{,}05; 0{,}95]$ (ref).

5. Metriche (`metrics.py`, `eval_evidence.py`)

$$\mathrm{P} = \frac{|\mathcal{S} \cap G|}{|\mathcal{S}|}, \qquad \mathrm{R} = \frac{|\mathcal{S} \cap G|}{|G|}, \qquad \mathrm{completo} = \mathbb{1}[\exists r : G^{(r)} \subseteq \mathcal{S}]$$

$$\text{sufficienza} = \mathbb{1}[p(\mathcal{S}) > 0{,}5], \qquad \text{necessità} = \mathbb{1}[p(\bar{\mathcal{S}}) \le 0{,}5]$$

$$\mathrm{BAcc} = \tfrac{1}{2}(\mathrm{TPR} + \mathrm{TNR}), \qquad \mathrm{ECE} = \sum_c \tfrac{|B_c|}{N}|\mathrm{acc}(B_c) - \mathrm{conf}(B_c)|$$

6. Dati (`make_splits.py`, `make_mix.py`, `baselines.py`)

$$u(doc) = \mathrm{sha256}(\texttt{"13:sale:doc_id"})_{[0:8]}/2^{64}, \qquad \text{negativi tenuti} = \mathrm{round}\Big(\frac{f\,P}{1 - f}\Big)$$

$$\mathrm{BM25}_k = \sum_w \mathrm{IDF}(w)\,\frac{\mathrm{tf}\,(k_1 + 1)}{\mathrm{tf} + k_1(1 - b + b\,|S_k|/\overline{|S|})}$$

7. Perché ogni scelta, in una riga

Scelta	Perché
un solo forward, modello congelato	latenza di un decision model (+0,5% misurato); verdetto invariato
preambolo con il claim	nel modello causale le frasi altrimenti non sanno che cosa si verifica
seconda copia del documento	ha già visto tutto il documento e il claim

Scelta	Perché
layer 19, 23, 27, 31	subito dopo un layer pieno; metà alta della rete; non solo l'ultimo, specializzato nella risposta
h^{end} e h^{mean}	fine della frase (vede il contesto) e media (non dipende dalla punteggiatura)
att_k	dove ha guardato il modello per decidere (E-ATT da solo: F1 0,75)
transformer non causale	la frase 0 vede la frase 40: evidenza sparsa
soglia $s_\star$ per documento, stessa w_o	numero di frasi adattato al documento; stessa scala dei punteggi
ZLPR	hinge morbido sul caso peggiore: completo vs non completo; gradiente concentrato su chi sbaglia
`mil_neg`	nessuna frase accesa sui claim non supportati
$\omega < 1$	annotazioni incomplete
split per documento con hash	nessuna fuga; stabile